

ĐẠI HỌC BÁCH KHOA HÀ NỘI
Trường Công nghệ Thông tin và Truyền thông



BÁO CÁO BÀI TẬP DỰ ÁN
PROJECT 1 IT3150

PHÂN TÍCH HÀNH VI HỌC SINH TRONG LỚP
HỌC SỬ DỤNG YOLOV12

Giáo viên hướng dẫn:
Sinh viên thực hiện:
MSSV:

Nguyễn Duy Hiệp
Trần Đình Tiến
20235229

Hà Nội, tháng 1 năm 2026

Mục lục

1	Mở đầu	4
1.1	Giới thiệu vấn đề	4
1.2	Mục tiêu nghiên cứu	4
2	Cơ sở lý thuyết	5
2.1	Tổng quan về Computer Vision và Video Processing Pipeline	5
2.2	YOLO (You Only Look Once)	5
2.3	Kiến trúc của YOLOv12	5
2.4	Phương pháp đánh giá mô hình	7
3	Phân tích bài toán và thiết kế hệ thống	8
3.1	Phân tích bài toán	8
3.2	Kiến trúc hệ thống	8
3.2.1	Multi-model architecture	8
3.2.2	Cải tiến hiệu chỉnh bằng confidence threshold tuning	9
3.2.3	Cải tiến khi xử lý video	9
3.2.4	Sơ đồ pipeline kiến trúc	9
4	Thu thập và xử lý dữ liệu	15
4.1	Thông tin dataset	15
4.2	Thông số huấn luyện	16
5	Thực nghiệm và kết quả	17
5.1	Môi trường thực nghiệm	17
5.1.1	Software dependencies	17
5.1.2	Nền tảng train model	17
5.2	Quá trình training và kết quả đánh giá	18
5.2.1	Training curves	18
5.2.2	Confusion matrices	26
5.2.3	Kết quả đánh giá chi tiết	30
5.2.4	So sánh giữa các models và repo trước	30
5.3	Cân bằng giữa độ chính xác và tốc độ	31
6	Hệ thống ứng dụng và demonstration	34
6.1	Kiến trúc ứng dụng	34
6.1.1	Core Modules	34
6.1.2	Application Modules	41
6.1.3	Entry Points	43
6.1.4	Luồng xử lý tổng thể	44
6.1.5	Điểm mạnh của kiến trúc	49
6.2	Tính năng và cấu hình	49
6.2.1	Các tính năng chính	49
6.2.2	Web UI với Gradio	50
6.2.3	Config parameters	50
6.3	Hướng dẫn sử dụng	51
6.3.1	Cài đặt và setup	51
6.3.2	Sử dụng Web UI	52
6.3.3	Live Camera Demo	53
6.3.4	Demo scenarios và sample outputs	54

7	Kết luận	56
7.1	Hạn chế của hệ thống	56
7.2	Hướng phát triển trong tương lai	56
	Tài liệu tham khảo	57

1 Mở đầu

1.1 Giới thiệu vấn đề

Trong bối cảnh giáo dục hiện đại, việc đánh giá trạng thái học tập và mức độ tương tác của từng học sinh trong lớp chủ yếu dựa vào quan sát chủ quan của giáo viên, dễ dẫn đến sai lệch và thiếu dữ liệu khách quan. Việc thu thập thông tin liên tục về hành vi (ví dụ: giơ tay, nhìn lên, cúi xuống, đứng dậy, quay lại) gặp nhiều thách thức do môi trường lớp học phức tạp: che khuất, nhiều đối tượng, góc nhìn hạn chế và biến động của tư thế cơ thể. Các phương pháp truyền thống (quan sát thủ công, ghi sổ) không đáp ứng yêu cầu phân tích liên tục, định lượng và có thể so sánh; trong khi các kỹ thuật thị giác máy tính đối mặt với hai vấn đề chính: Thiếu bộ dữ liệu chuyên biệt, chú thích đầy đủ cho hành vi lớp học; Khó khăn khi chỉ dựa vào toạ độ khung xương (skeleton) trong điều kiện che khuất hoặc mất thông cảnh nền. Do đó cần một giải pháp tự động, khách quan để nhận diện, đếm và phân tích hành vi học sinh trong lớp, đồng thời cung cấp chỉ số đánh giá hiệu quả giảng dạy dựa trên dữ liệu quan sát được

1.2 Mục tiêu nghiên cứu

Mục tiêu chung

Phát triển một phương pháp thông minh để nhận diện, định vị, đếm và phân tích hành vi học sinh trong lớp học, từ đó cung cấp dữ liệu khách quan phục vụ đánh giá hiệu quả giảng dạy.

Mục tiêu cụ thể

- Sử dụng 1 bộ dữ liệu để phân tích hành vi lớp học, bao gồm chú thích các hình ảnh minh họa cho 5 hành vi phổ biến (giơ tay, nhìn quanh, nhìn xuống, nhìn lên, nghe giảng, đứng,...)
- Ứng dụng mô hình nhận diện đối tượng (YOLO) để phân loại và đếm các hành vi của học sinh trong suốt thời gian tiết học, đạt độ chính xác nhận diện và định vị ở mức cao
- Thiết kế và tích hợp hệ số trọng số (Nonlinear Weighting Factor) cho các hành vi trong các kịch bản lớp học khác nhau, phục vụ đánh giá hiệu quả giảng dạy dựa trên mức độ tập trung (attention) và hoạt động (activity) của học sinh
- Xây dựng khung phân tích và báo cáo kết quả để cung cấp dữ liệu khách quan cho giáo viên và nhà quản lý giáo dục, làm nền tảng cho các phương pháp đánh giá lớp học thông minh

2 Cơ sở lý thuyết

2.1 Tổng quan về Computer Vision và Video Processing Pipeline

Computer Vision là lĩnh vực khoa học máy tính đóng vai trò là đôi mắt và bộ não của hệ thống trí tuệ nhân tạo, cho phép máy tính có khả năng thu nhận, xử lý và hiểu được các dữ liệu thị giác từ thế giới thực. Thay vì chỉ coi hình ảnh là các ma trận điểm ảnh (pixels) vô hồn, thị giác máy tính sử dụng các mô hình toán học và học sâu (Deep Learning) để trích xuất các thông tin có ý nghĩa, từ việc nhận diện màu sắc, hình dạng cho đến việc hiểu được ngữ cảnh phức tạp của một bức ảnh.

Object Detection là một kỹ thuật then chốt trong thị giác máy tính, kết hợp giữa hai nhiệm vụ: phân loại (Classification) và định vị (Localization). Trong khi phân loại chỉ xác định sự hiện diện của vật thể, thì phát hiện đối tượng sẽ xác định chính xác vị trí của từng đối tượng đó trong khung hình thông qua các khung bao (Bounding Boxes). Đây là bước quan trọng để hệ thống không chỉ biết có gì mà còn biết ở đâu, tạo tiền đề cho việc tương tác hoặc theo dõi vật thể trong không gian.

Video Processing Pipeline là một quy trình kiến trúc bao gồm chuỗi các giai đoạn kế tiếp nhau để xử lý dữ liệu video theo thời gian thực. Một video thực chất là tập hợp hàng loạt các khung hình tĩnh chuyển động nhanh, vì vậy hệ thống cần một dây chuyền vận hành chặt chẽ từ khâu nạp dữ liệu, tiền xử lý, đưa qua mô hình AI để nhận diện, cho đến khâu hậu xử lý và hiển thị kết quả. Hiệu năng của một Pipeline được đánh giá dựa trên khả năng duy trì tốc độ xử lý mượt mà (FPS) mà vẫn đảm bảo độ chính xác cho từng khung hình khi luân chuyển qua hệ thống.

2.2 YOLO (You Only Look Once)

YOLO là một kiến trúc mạng thần kinh nhân tạo đột phá trong bài toán phát hiện đối tượng, được giới thiệu lần đầu vào năm 2015. Khác với các phương pháp truyền thống trước đó (như R-CNN) thường chia bài toán làm hai giai đoạn riêng biệt: đề xuất vùng chứa vật thể rồi mới phân loại, **YOLO** tiếp cận theo hướng một giai đoạn (one-stage). Đúng như tên gọi của mình, thuật toán này chỉ nhìn một lần duy nhất qua toàn bộ bức ảnh để dự đoán đồng thời cả vị trí khung bao (bounding box) và xác suất của các lớp đối tượng, giúp giảm đáng kể chi phí tính toán.

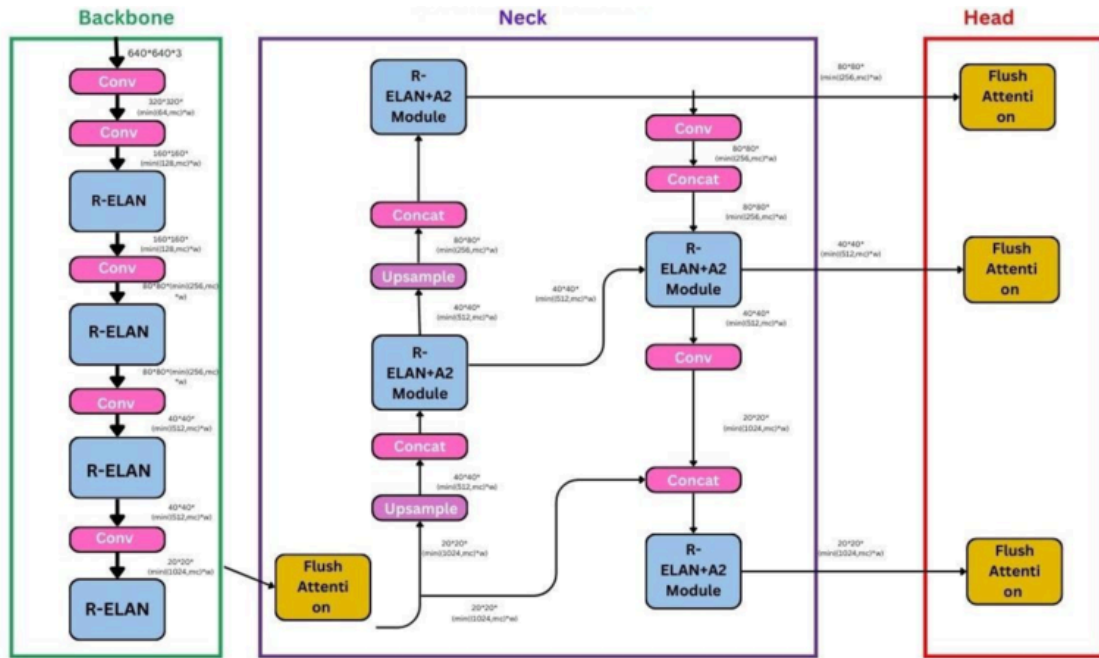
Về mặt kỹ thuật, **YOLO** chia hình ảnh đầu vào thành một lưới các ô. Mỗi ô này chịu trách nhiệm dự đoán các khung bao và điểm tin cậy (confidence scores) cho những đối tượng nằm trong phạm vi của nó. Nhờ việc xử lý toàn bộ hình ảnh trong một lượt truyền thẳng (forward pass) duy nhất qua mạng CNN, **YOLO** có thể tận dụng thông tin ngữ cảnh toàn cục để tránh các sai sót phổ biến như nhận diện nhầm các mảng nền là đối tượng. Điều này giúp mô hình đạt được tốc độ xử lý cực nhanh, vượt xa tốc độ thực tế của nhiều thuật toán cùng thời.

Chính nhờ sự cân bằng xuất sắc giữa độ chính xác và tốc độ, **YOLO** trở thành lựa chọn ưu tiên hàng đầu trong các Video Processing Pipeline yêu cầu tính thời gian thực. Từ phiên bản gốc đến các biến thể hiện đại sau này, **YOLO** không ngừng được cải tiến để hoạt động hiệu quả trên cả các thiết bị phần cứng có tài nguyên hạn chế như camera thông minh hay thiết bị di động, đóng vai trò là trái tim của nhiều hệ thống giám sát và phân tích video tự động hiện nay.

2.3 Kiến trúc của YOLOv12

YOLO12 tích hợp một số cải tiến quan trọng để cân bằng giữa tốc độ và độ chính xác. Cơ chế Attention Area xử lý hiệu quả các trường tiếp nhận lớn, giảm chi phí tính toán so với self-attention tiêu chuẩn. Residual Efficient Layer Aggregation Networks (R-ELAN) cải thiện việc tổng hợp

đặc trưng, giải quyết các thách thức tối ưu hóa trong các mô hình tập trung vào attention lớn hơn. Optimized Attention Architecture, bao gồm việc sử dụng FlashAttention và loại bỏ positional encoding, giúp tăng cường hơn nữa hiệu quả. Các tính năng này cho phép **YOLOv12** đạt được độ chính xác hàng đầu trong khi vẫn duy trì tốc độ suy luận theo thời gian thực, điều này rất quan trọng đối với nhiều ứng dụng.



Hình 1: Kiến trúc chi tiết của YOLOv12

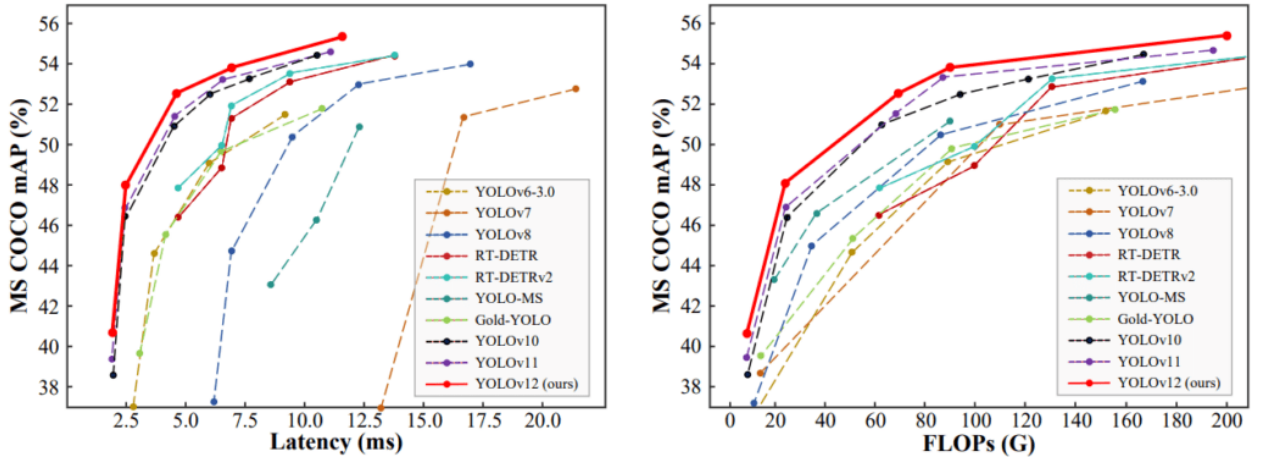
YOLOv12 được xây dựng theo kiến trúc ba phần chính gồm Backbone, Neck và Head, kế thừa cấu trúc đặc trưng của các mô hình YOLO một giai đoạn, nhằm đảm bảo cân bằng giữa độ chính xác và tốc độ xử lý trong bài toán phát hiện đối tượng thời gian thực.

Phần Backbone có nhiệm vụ trích xuất đặc trưng từ ảnh đầu vào. Backbone sử dụng xen kẽ các lớp Convolution và khối R-ELAN để giảm dần kích thước không gian của ảnh trong khi tăng số kênh đặc trưng. Các khối R-ELAN giúp tổng hợp thông tin hiệu quả hơn, cải thiện khả năng học đặc trưng và duy trì dòng gradient ổn định, từ đó tạo ra các feature map ở nhiều mức độ phân giải khác nhau phục vụ cho phát hiện đa tỉ lệ.

Neck của **YOLOv12** được thiết kế để kết hợp và tinh chỉnh các đặc trưng trích xuất từ backbone. Mô hình sử dụng các mô-đun R-ELAN+A2 kết hợp với các phép Upsample và Concat nhằm hợp nhất thông tin ngữ nghĩa sâu từ các tầng dưới với thông tin không gian chi tiết từ các tầng trên. Quá trình này giúp tăng cường khả năng phát hiện các đối tượng có kích thước nhỏ và trung bình. Ngoài ra, cơ chế Flush Attention được tích hợp trong neck để làm nổi bật các đặc trưng quan trọng và giảm ảnh hưởng của nhiễu.

Phần Head chịu trách nhiệm đưa ra các dự đoán cuối cùng của mô hình, bao gồm vị trí bounding box, độ tin cậy và nhãn lớp. Các feature map đầu ra tại nhiều mức độ phân giải khác nhau được xử lý song song, kết hợp với Flush Attention nhằm tăng khả năng tập trung vào các vùng chứa đối tượng, giúp mô hình phát hiện hiệu quả các đối tượng có kích thước khác nhau trong cùng một ảnh.

Tổng thể, kiến trúc **YOLOv12** cho thấy sự cải tiến trong việc trích xuất và kết hợp đặc trưng thông qua R-ELAN, R-ELAN+A2 và Flush Attention, đồng thời vẫn giữ được ưu điểm về tốc độ suy luận, phù hợp cho các ứng dụng phát hiện đối tượng trong thời gian thực.



Hình 2: So sánh hiệu suất giữa YOLOv12 và các phiên bản trước đó

2.4 Phương pháp đánh giá mô hình

Trong bài toán phát hiện đối tượng, việc đánh giá hiệu năng mô hình không chỉ dựa vào việc dự đoán đúng lớp mà còn phụ thuộc vào độ chính xác của vị trí khung bao thông qua chỉ số *IoU* (Intersection over Union).

- **Precision** (Độ chính xác) đo lường khả năng tránh dự đoán nhầm các đối tượng không tồn tại

$$Precision = \frac{TP}{TP + FP}$$

- **Recall** (Độ triệu hồi) đo lường khả năng không bỏ sót các đối tượng thực tế

$$Recall = \frac{TP}{TP + FN}$$

- **F1** được sử dụng như một trung bình điều hòa để đại diện cho sự cân bằng giữa Precision và Recall

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

- **mAP** (Mean Average Precision) là chỉ số bao quát nhất, được tính bằng cách lấy trung bình các giá trị Precision ở các mức Recall khác nhau (diện tích dưới đường cong Precision-Recall) cho tất cả các lớp đối tượng, giúp cung cấp một cái nhìn tổng thể về độ tin cậy và sức mạnh của mô hình trên toàn bộ tập dữ liệu

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i$$

Trong đó:

- N : Tổng số lớp đối tượng trong mô hình
- TP (True Positive): Dự đoán đúng đối tượng thực tế
- FP (False Positive): Dự đoán sai đối tượng

- FN (False Negative): Bỏ sót đối tượng thực tế
- AP_i : Độ chính xác trung bình của lớp thứ i

3 Phân tích bài toán và thiết kế hệ thống

3.1 Phân tích bài toán

Bài toán được đưa ra nhằm mục tiêu tự động phát hiện và phân loại hành vi của học sinh trong lớp học dựa trên dữ liệu hình ảnh. Hệ thống sẽ quan sát hoạt động của học sinh và xác định hành vi đang diễn ra tại từng thời điểm, hỗ trợ giáo viên trong việc:

- Theo dõi mức độ tham gia học tập
- Đánh giá tương tác trong lớp
- Phát hiện các hành vi bất thường hoặc thiếu tập trung

Hệ thống tập trung vào các loại hành vi phổ biến sau:

STT	Hành vi	Mô tả
1	Hand-raise	Giơ tay
2	Read	Đọc sách
3	Write	Ghi chép
4	Talk	Nói chuyện
5	Stand	Đứng lên

Bài toán gặp nhiều thách thức trong thực tế, cụ thể:

- Nhiều học sinh trong một khung hình, dẫn đến che khuất
- Hành vi có sự tương đồng cao như *Read* và *Write*
- Hành vi mang tính động, cần được xem xét chuỗi thời gian, không chỉ 1 frame, một hành vi diễn ra trong nhiều frame liên tiếp
- Điều kiện môi trường lớp học: ánh sáng thay đổi, khoảng cách xa – học sinh nhỏ trong khung hình,...
- Dữ liệu gán nhãn hạn chế: khó thu thập và gán nhãn chính xác, mất cân bằng giữa các lớp hành vi

3.2 Kiến trúc hệ thống

3.2.1 Multi-model architecture

Vì các dataset tìm kiếm được hoàn toàn độc lập, không liên quan đến nhau nên các nhãn hành vi không được gán một cách nhất quán, dẫn đến lẫn lộn nhãn khi huấn luyện trên cùng một model. Do đó, cần finetune độc lập 3 model tương ứng với 3 dataset tìm được, sau đó khi inference sẽ sử dụng song song 3 model, cụ thể:

- Dataset chứa nhãn hand-raise, read, write: 1 model finetune từ YOLOv12

- Dataset chứa nhãn `stand`: 1 model finetune từ YOLOv12
- Dataset chứa nhãn `talk`: 1 model finetune từ YOLOv12

3.2.2 Cải tiến hiệu chỉnh bằng confidence threshold tuning

Vì YOLO luôn dự đoán rất nhiều bounding box, và không phải box nào cũng đáng tin, dẫn đến hàng trăm box rác xuất hiện, gây dư thừa, sai lệch kết quả. Do đó cần bộ lọc kiểm soát box nào đáng tin, box nào cần bỏ qua.

Confidence threshold tuning ở đây sử dụng một ngưỡng giới hạn điểm confidence tối thiểu chấp nhận, được hiệu chỉnh theo tính chất của các nhãn. Ví dụ: nhãn `write` và `read` rất dễ nhầm nên cần đặt confidence threshold cao hơn.

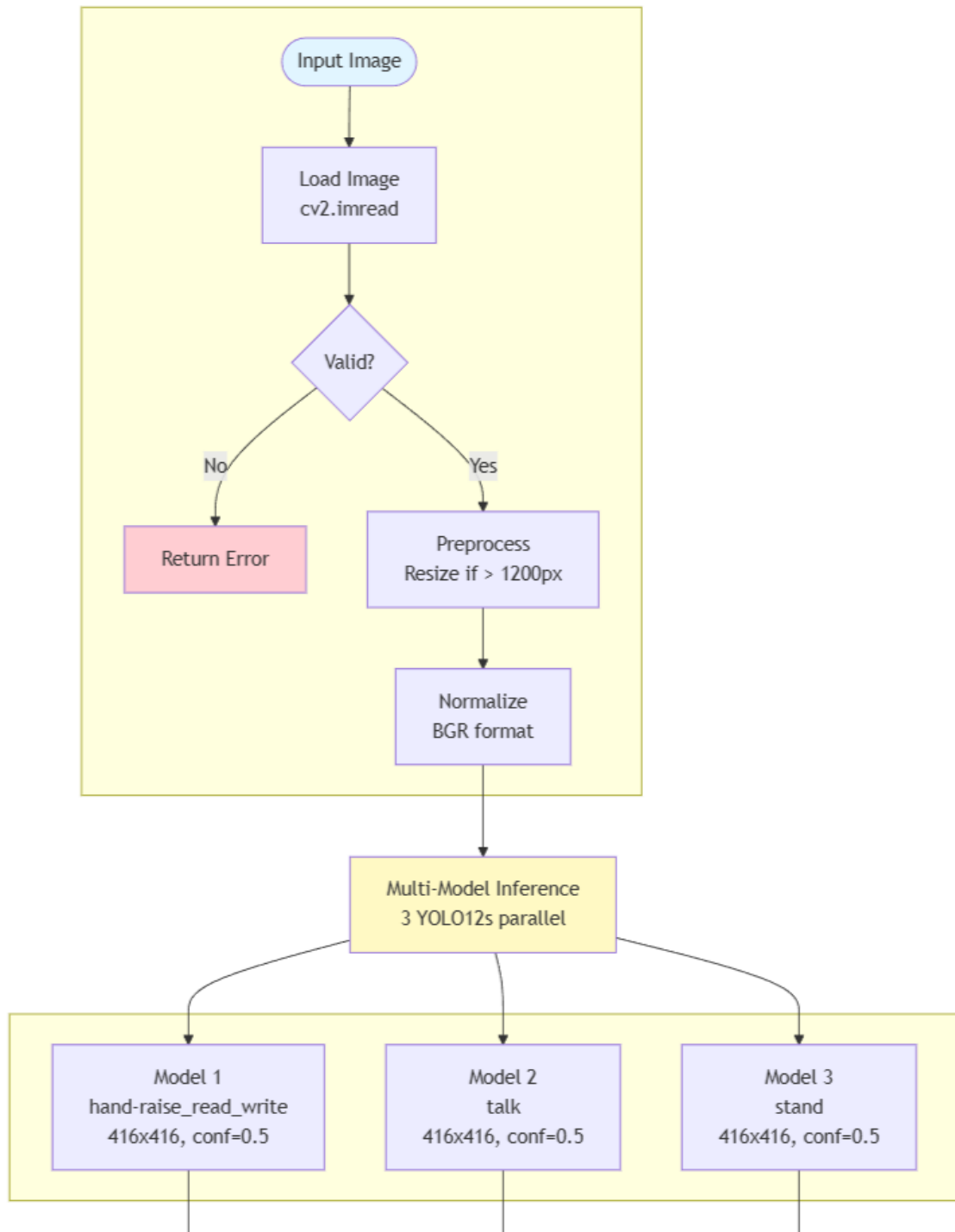
3.2.3 Cải tiến khi xử lý video

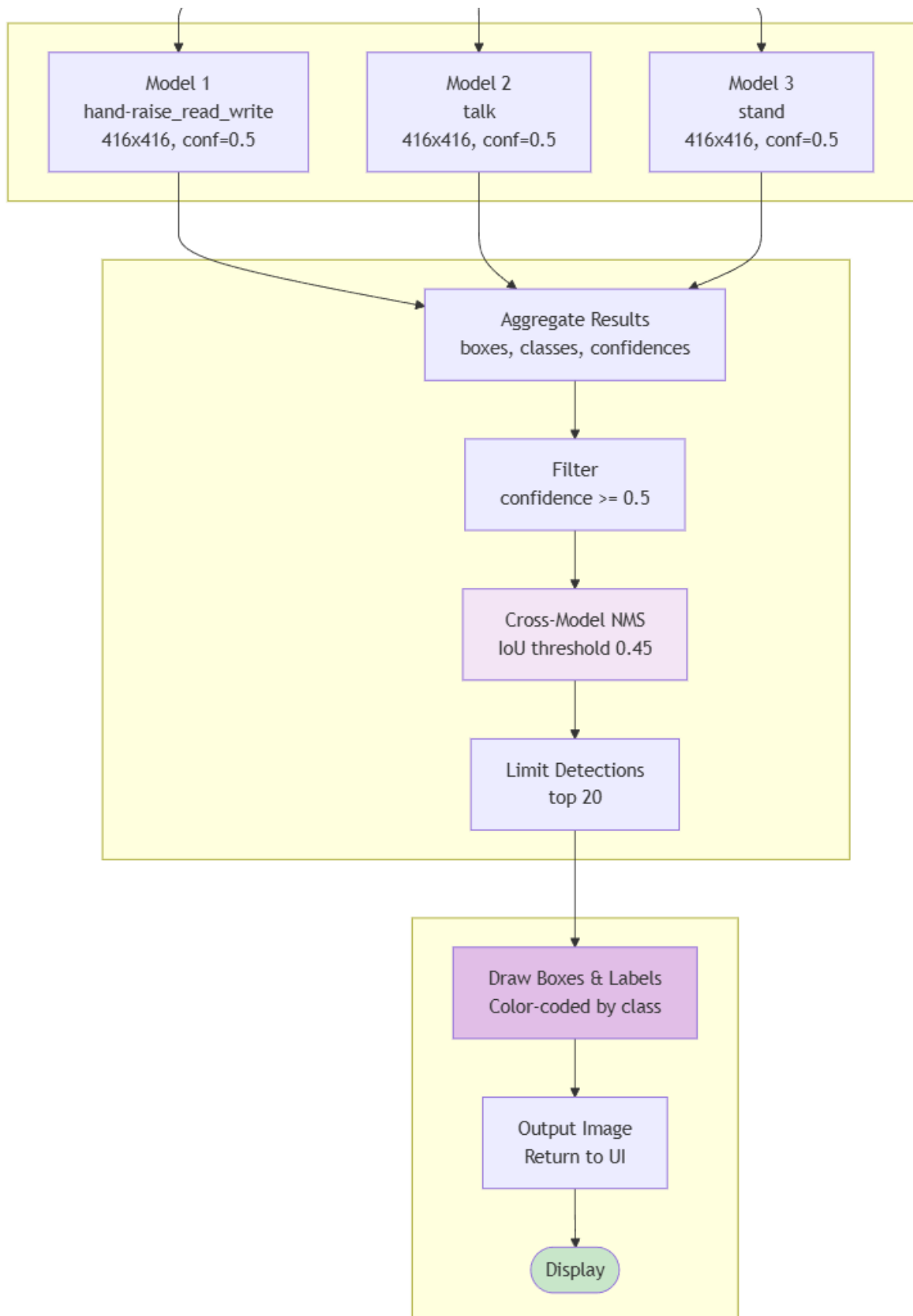
Video chứa nhiều frame, các frame liên kế rất giống nhau, gây tốn thời gian xử lý. Do đó, ta không xử lý mọi frame mà dùng kỹ thuật frame skipping, tức bỏ frame theo chu kỳ. Ít frame hơn giúp giảm thời gian load của CPU/GPU.

Ví dụ: giả sử đầu vào video 30 FPS

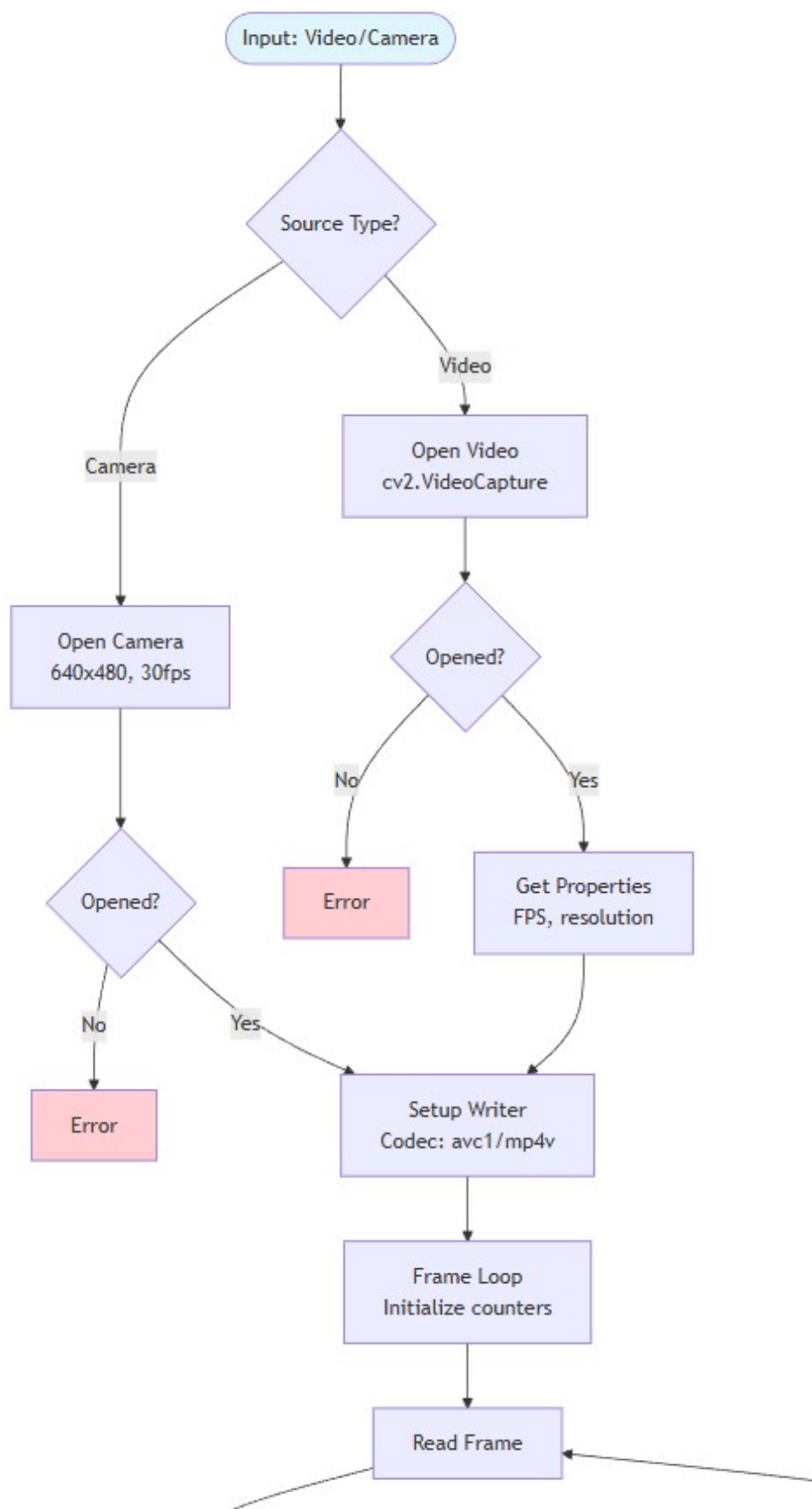
- Với $skip = 1$ thì chỉ xử lý 15 FPS (tuần tự lấy 1 frame bỏ 1 frame)
- Với $skip = 2$ thì chỉ xử lý 10 FPS (tuần tự lấy 1 frame bỏ 2 frame)

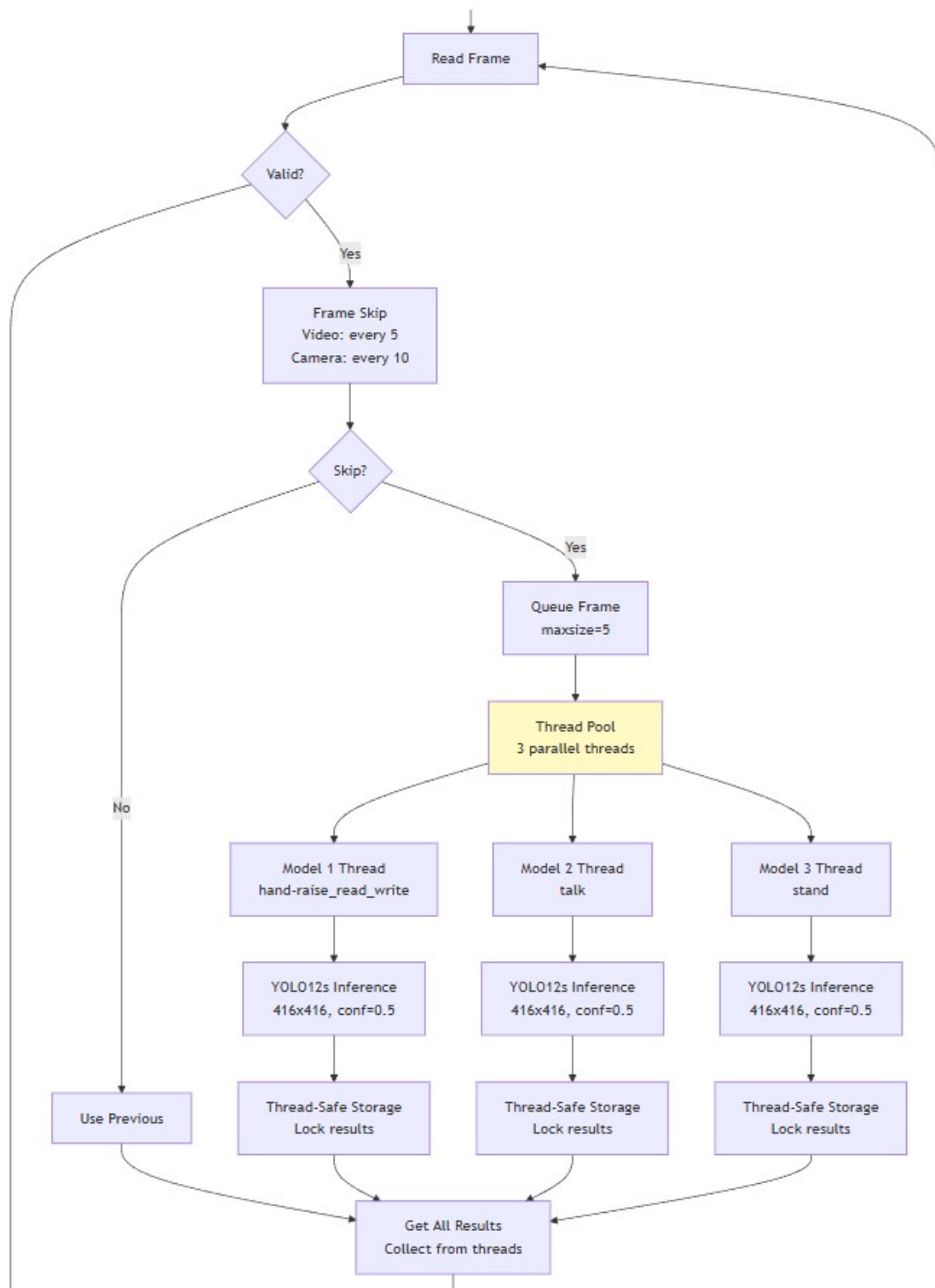
3.2.4 Sơ đồ pipeline kiến trúc

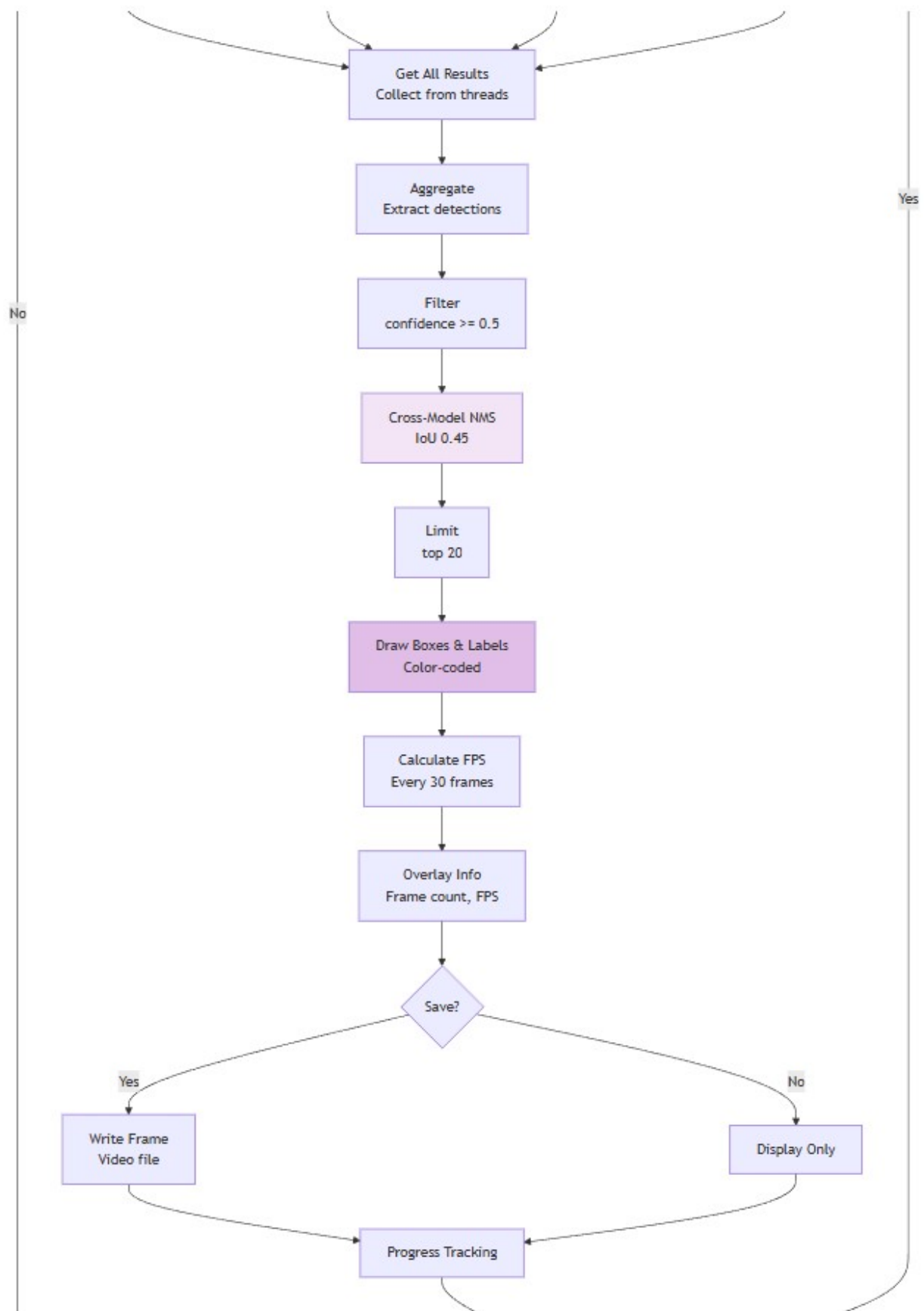


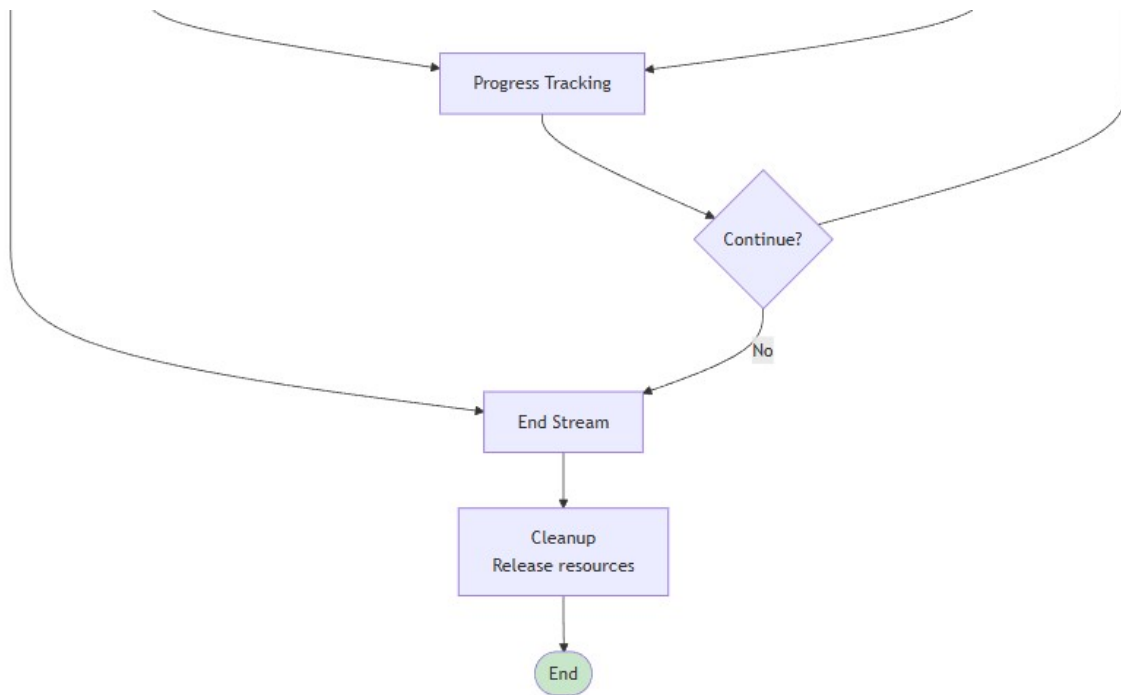


Hình 3: Xử lý ảnh









Hình 4: Xử lý video

4 Thu thập và xử lý dữ liệu

4.1 Thông tin dataset

Dataset chứa nhãn hand-raise, read, write:

- Link: <https://www.kaggle.com/datasets/nonpat/detecting-student-classroom-behavior2>
- Số lượng mẫu dữ liệu: ≈ 3000 ảnh
- Cấu trúc thư mục:

```

├── data_test2/
│   ├── images/
│   │   ├── test/ ..... *.jpg
│   │   ├── train/ ..... *.jpg
│   │   └── val/ ..... *.jpg
│   ├── labels/
│   │   ├── test/ ..... *.txt
│   │   ├── train/ ..... *.txt
│   │   └── val/ ..... *.txt
│   └── config.yaml
  
```

Dataset chứa nhãn stand:

- Link: <https://www.kaggle.com/datasets/shreyasudaya/scb-05-dataset>
- Số lượng mẫu dữ liệu: ≈ 6000
- Cấu trúc thư mục:

```
├── images/
│   ├── train/ ..... *.jpg
│   └── val/ ..... *.jpg
├── labels/
│   ├── train/ ..... *.txt
│   └── val/ ..... *.txt
└── config.yaml
```

Dataset chứa nhãn talk:

- Link: <https://www.kaggle.com/datasets/shreyasudaya/scb-05-dataset>
- Số lượng mẫu dữ liệu: ≈ 4000
- Cấu trúc thư mục:

```
├── images/
│   ├── train/ ..... *.jpg
│   └── val/ ..... *.jpg
├── labels/
│   ├── train/ ..... *.txt
│   └── val/ ..... *.txt
└── config.yaml
```

Mỗi bounding box label thật sẽ ứng với 1 dòng trong file .txt, mỗi dòng đó có dạng:

< class_id > < x_center > < y_center > < width > < height >

Trong đó:

- *class_id*: Số nguyên bắt đầu từ 0 ứng với số hiệu nhãn
- *x_center*: Tọa độ tâm box theo trục *x*
- *y_center*: Tọa độ tâm box theo trục *y*
- *width*: Chiều rộng bounding box
- *height*: Chiều cao bounding box

4.2 Thông số huấn luyện

- Số epoch: 90 - 100
- Kích thước một batch: 8 ảnh
- Tỷ lệ train/val/test: chia sẵn trong dataset

5 Thực nghiệm và kết quả

5.1 Môi trường thực nghiệm

5.1.1 Software dependencies

Hệ thống được phát triển trên Python 3.8+ với các thư viện chính:

- **Ultralytics YOLOv12**: Framework cho training và inference các mô hình YOLO
- **OpenCV**: Xử lý ảnh/video và camera
- **Gradio**: Giao diện web
- **NumPy**: Tính toán số học, xử lý ma trận
- **Pandas**: Phân tích dữ liệu

Các dependencies khác được quản lý qua `requirements.txt`.

5.1.2 Nền tảng train model

Quá trình huấn luyện các mô hình được thực hiện trên Google Colab với GPU miễn phí, sau đó tải các file weights (.pt) về máy local để sử dụng trong ứng dụng. Cấu hình training:

- Model: YOLO12s (YOLO version 12, small variant)
- Epochs:
 - + Model hand-raise_read_write và talk: 100 epochs
 - + Model stand: 90 epochs (điều chỉnh để tránh quá giới hạn runtime của Google Colab)
- Batch size: 8
- Image size: 640x640 pixels
- Learning rate: 0.01 (initial), cosine schedule
- Optimizer: SGD (momentum 0.937, weight decay 0.0005)
- Data augmentation: Mosaic, flip horizontal, HSV, auto-augment, erasing

Sau khi training hoàn tất trên Colab, các file best.pt được tải xuống và lưu tại:

```
modules/models/student_behavior
├── hand-raise_read_write_model
│   └── weights/best.pt
├── talk_model
│   └── weights/best.pt
└── stand_model
    └── weights/best.pt
```

5.2 Quá trình training và kết quả đánh giá

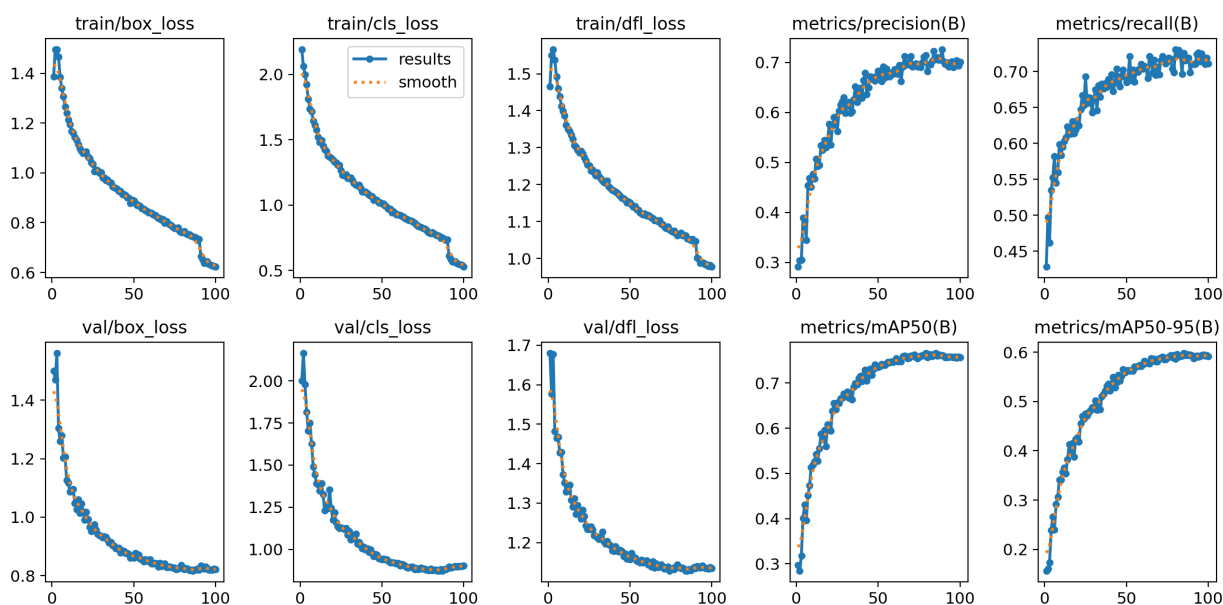
5.2.1 Training curves

Quá trình huấn luyện được theo dõi thông qua các training curves, thể hiện sự thay đổi của loss và metrics qua các epochs.

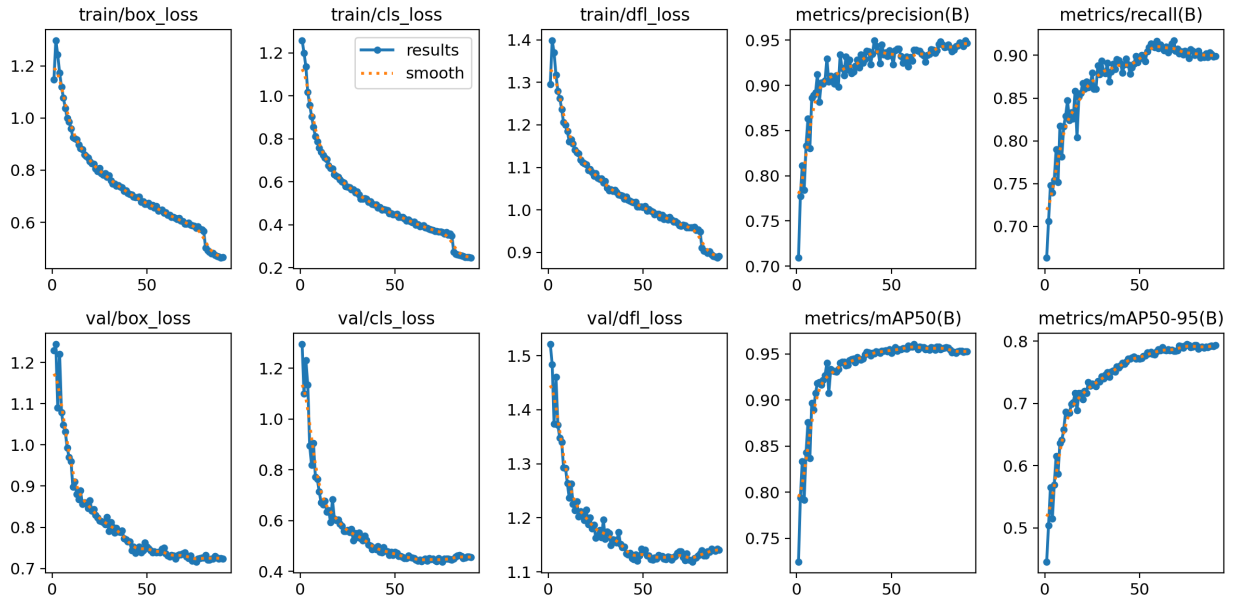
Loss curves

Các loss curves bao gồm:

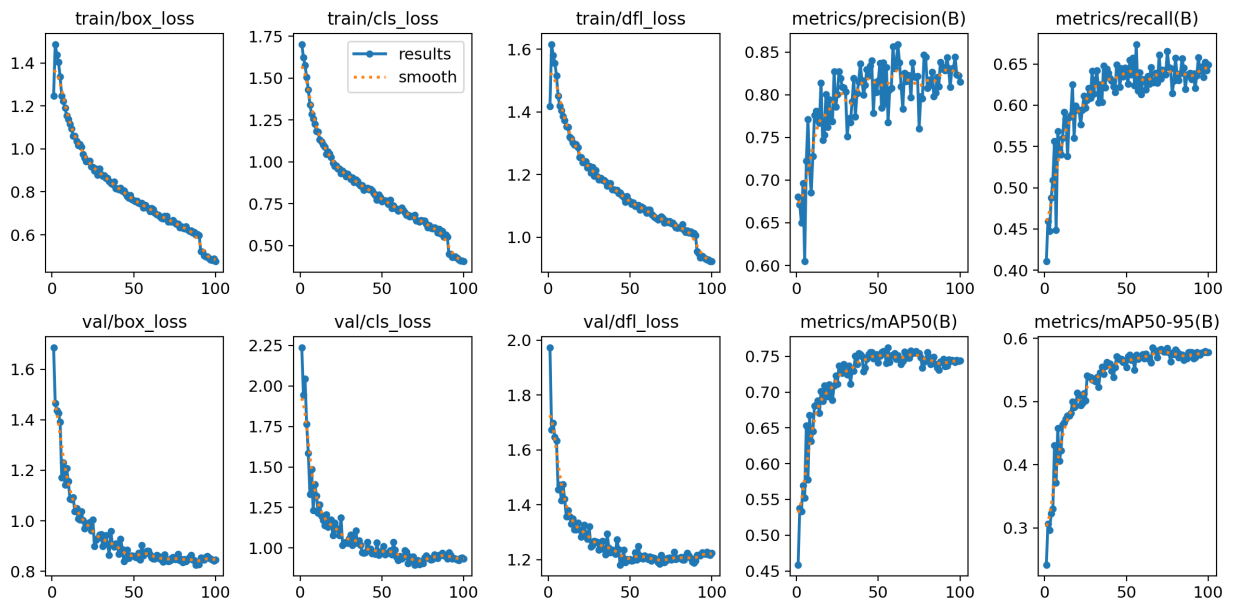
- `train/box_loss`: Loss cho bounding box regression, giảm dần qua các epochs
- `train/cls_loss`: Loss cho classification, giảm dần khi model học phân loại tốt hơn
- `train/dfl_loss`: Distribution Focal Loss, giảm dần
- `val/box_loss`, `val/cls_loss`, `val/dfl_loss`: Validation losses, thường cao hơn train loss một chút



Hình 5: Results của `hand-raise_read_write_model`



Hình 6: Results của `talk_model`

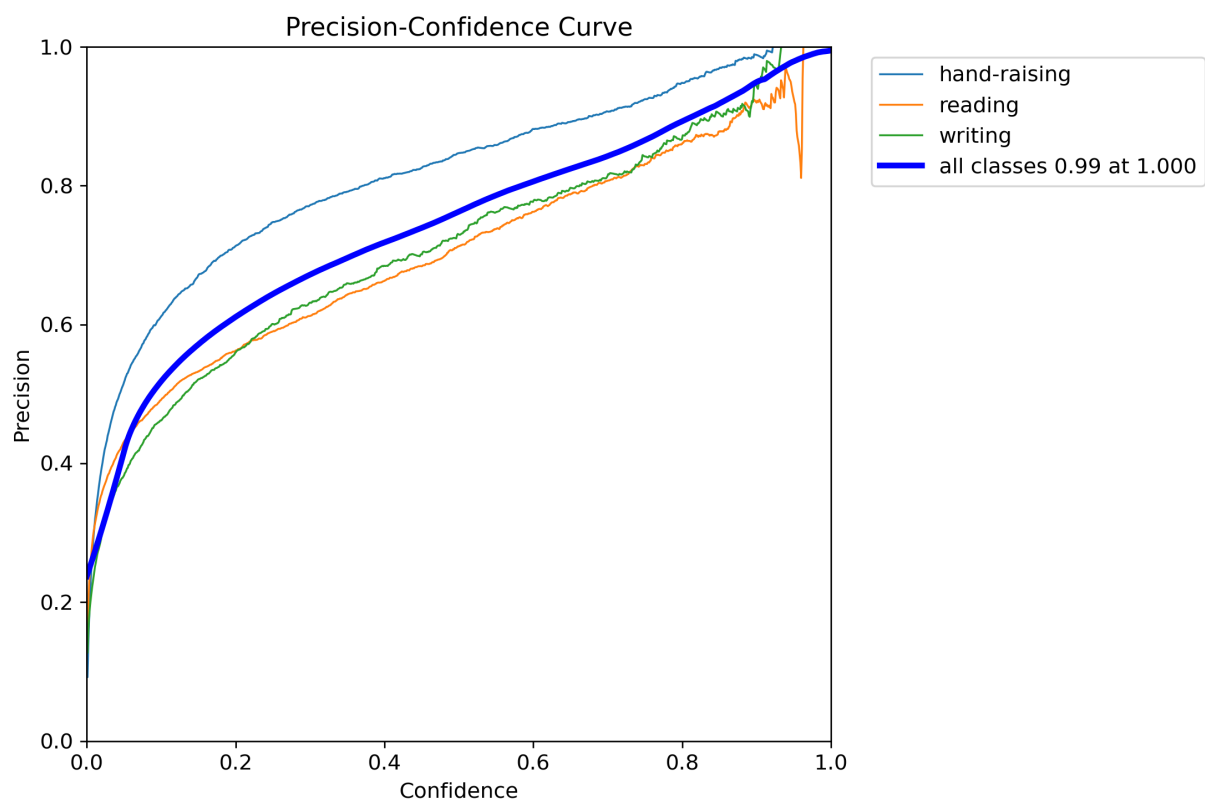


Hình 7: Results của `stand_model`

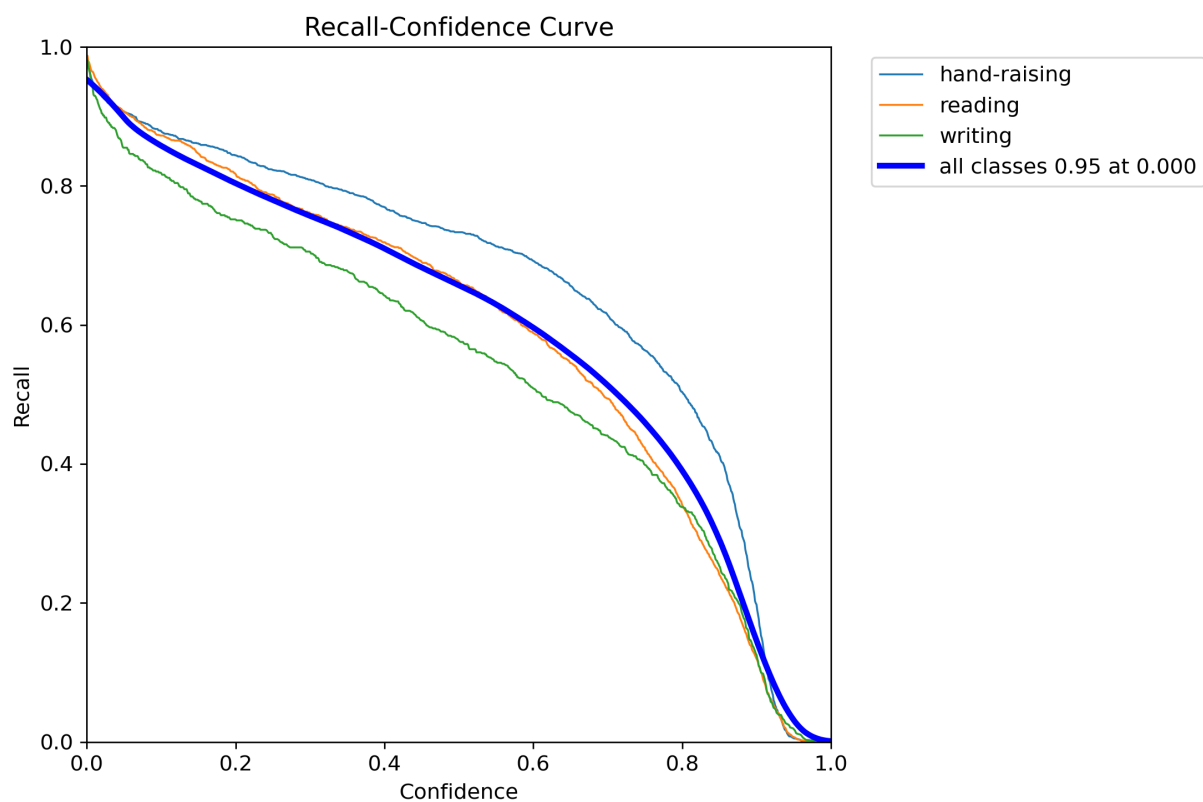
Metrics curves

Các metrics curves bao gồm:

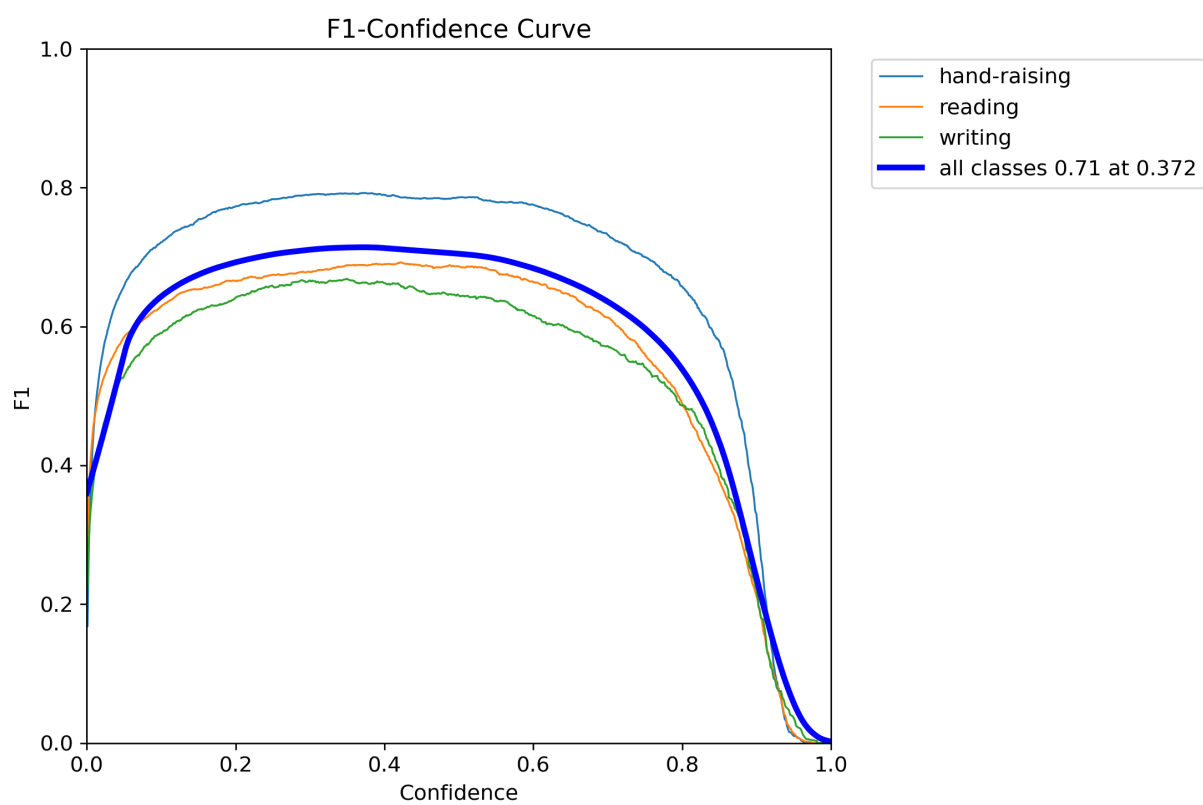
- `metrics/precision(B)`: *Precision* trên validation set, tăng dần qua các epochs
- `metrics/recall(B)`: *Recall* trên validation set, tăng dần
- `metrics/mAP50(B)`: *Mean Average Precision* tại *IoU* threshold 0.5
- `metrics/mAP50-95(B)`: *Mean Average Precision* trung bình từ *IoU* 0.5 đến 0.95



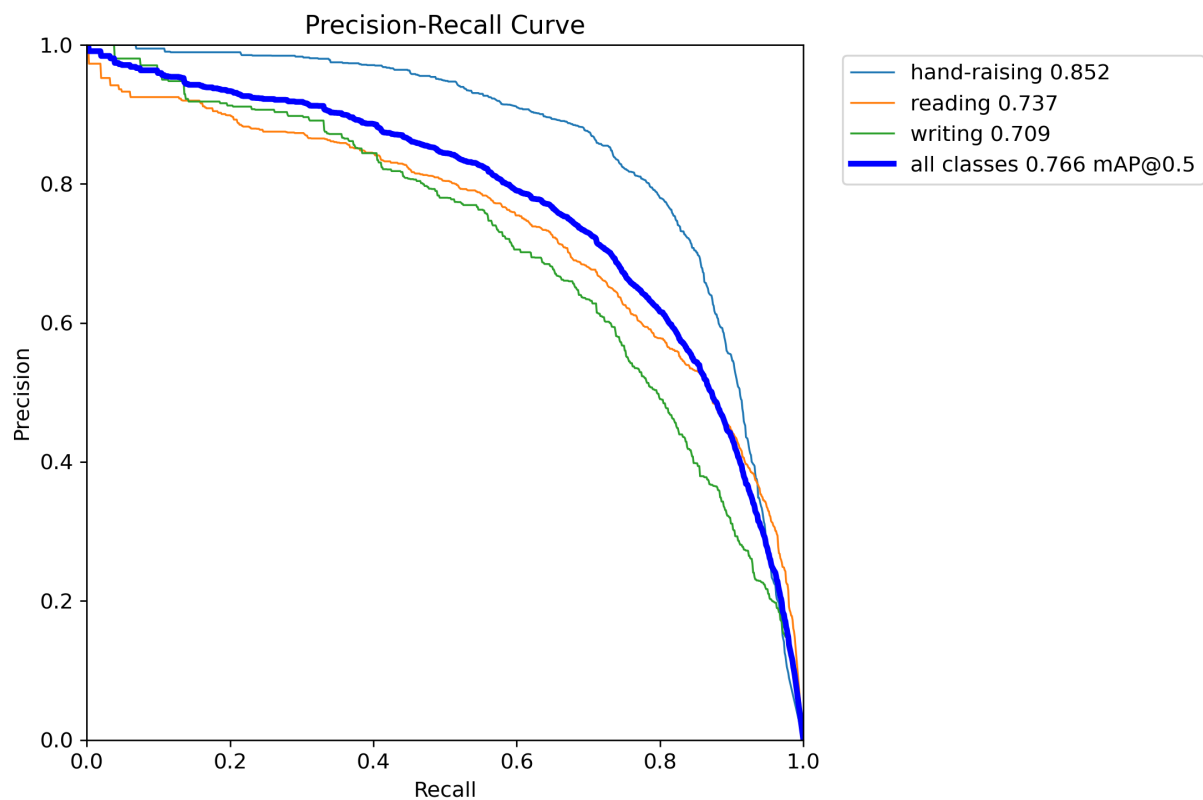
Hình 8: BoxP_curve của hand-raise_read_write_model



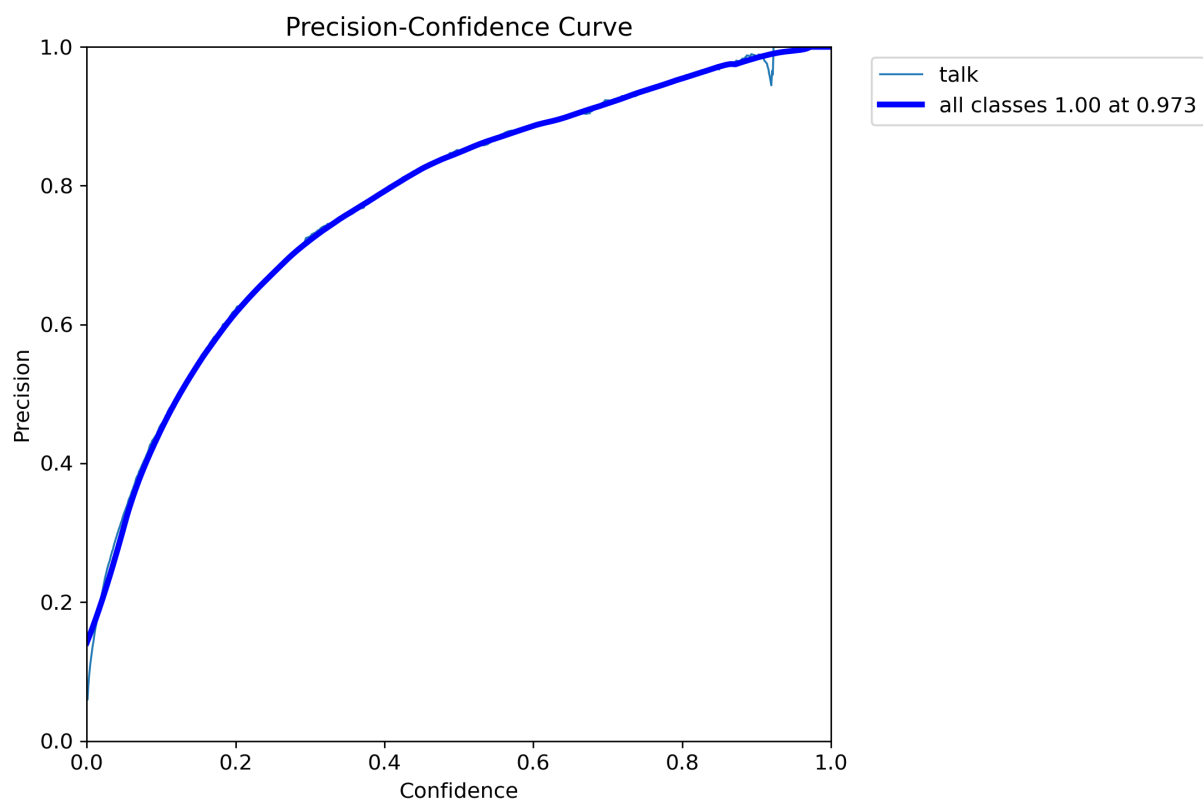
Hình 9: BoxR_curve của hand-raise_read_write_model



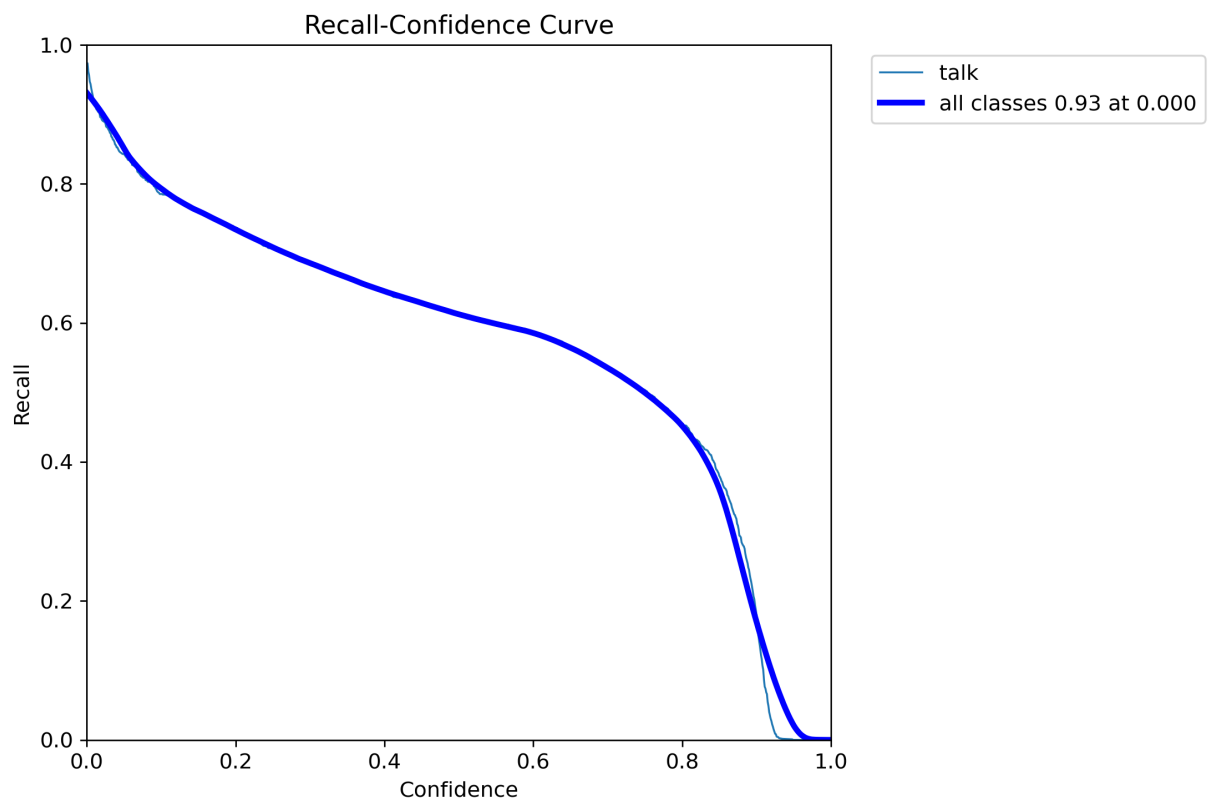
Hình 10: BoxF1_curve của hand-raise_read_write_model



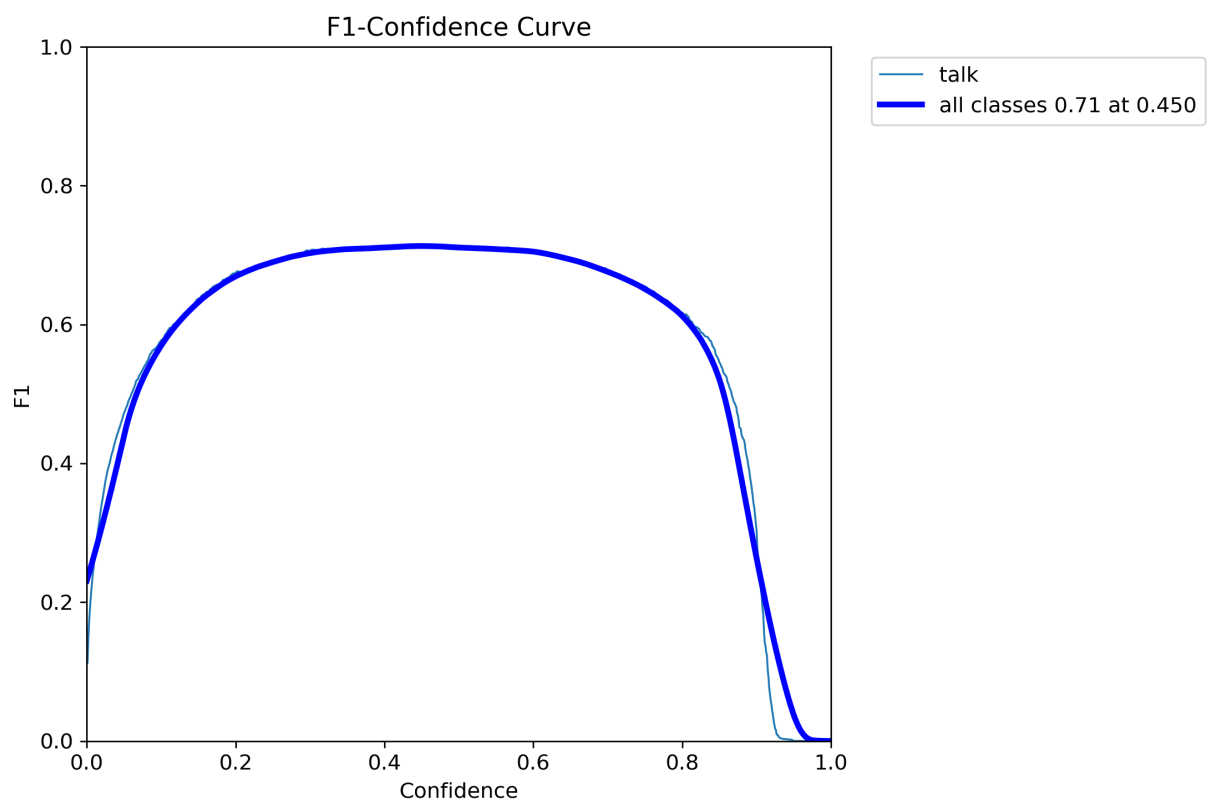
Hình 11: BoxPR_curve của hand-raise_read_write_model



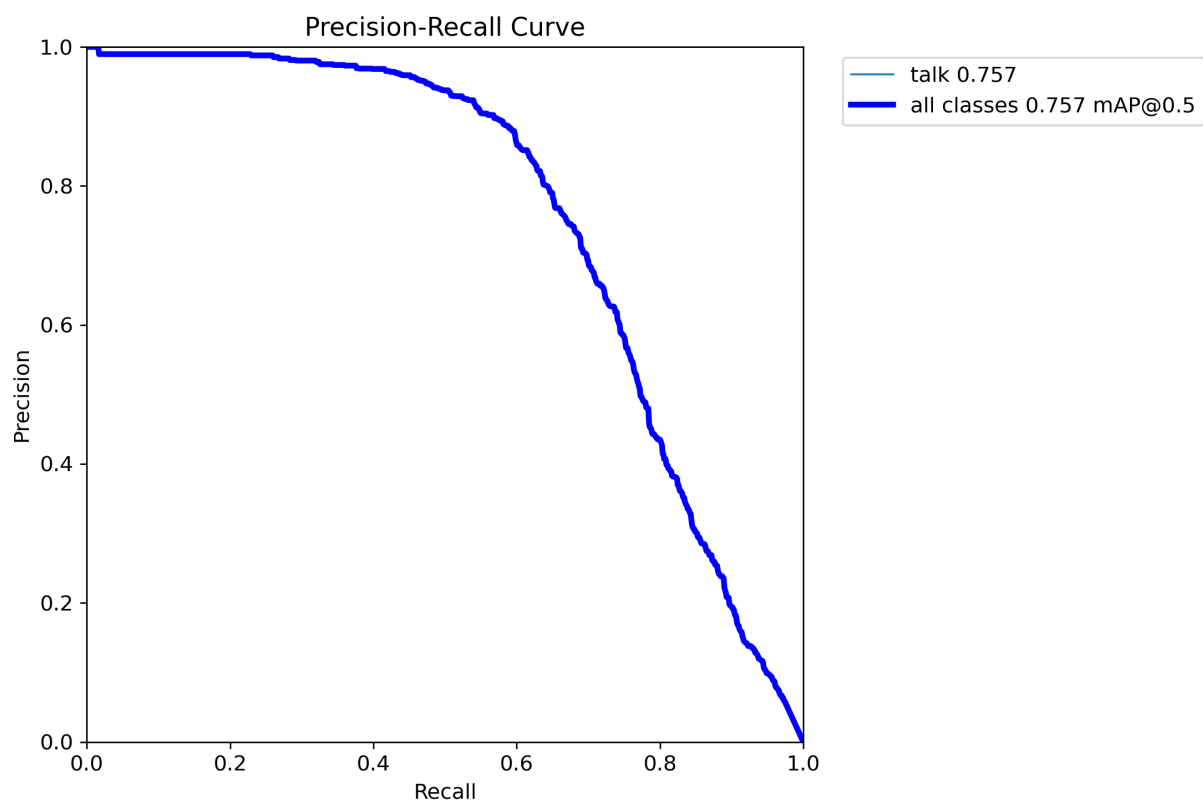
Hình 12: BoxP_curve của talk_model



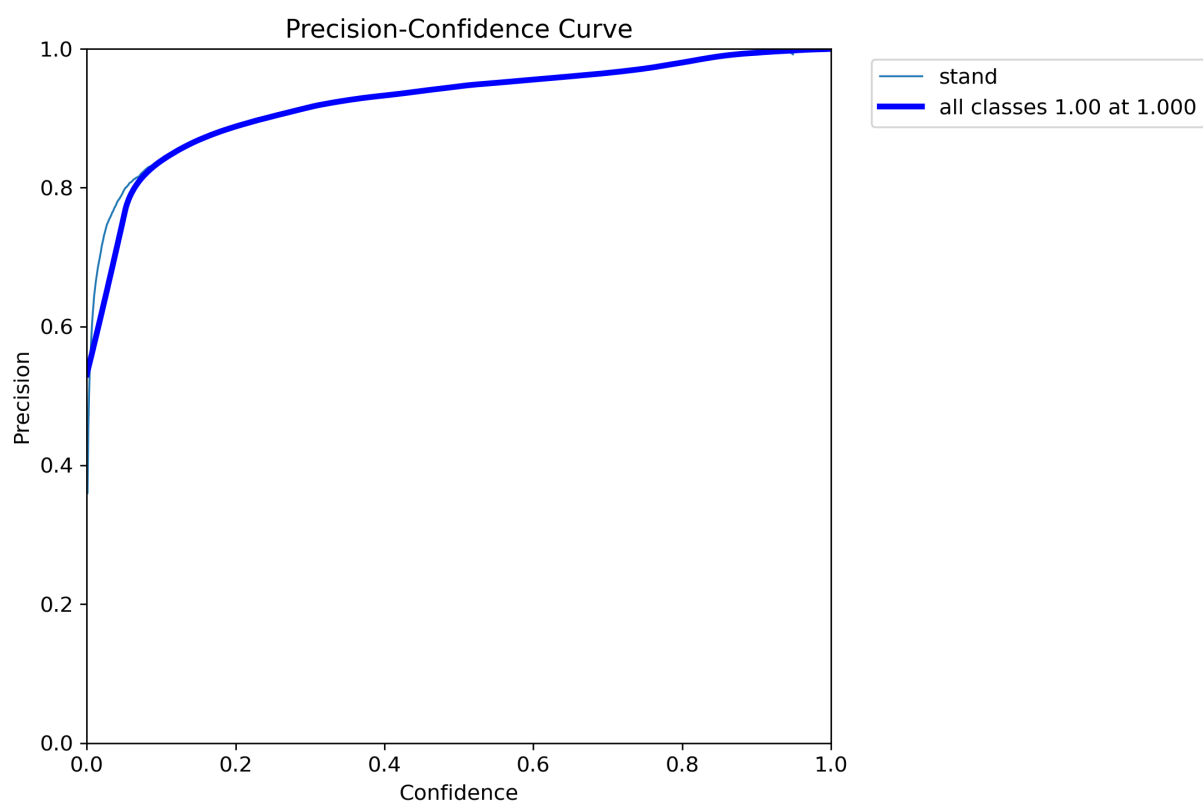
Hình 13: BoxR_curve của talk_model



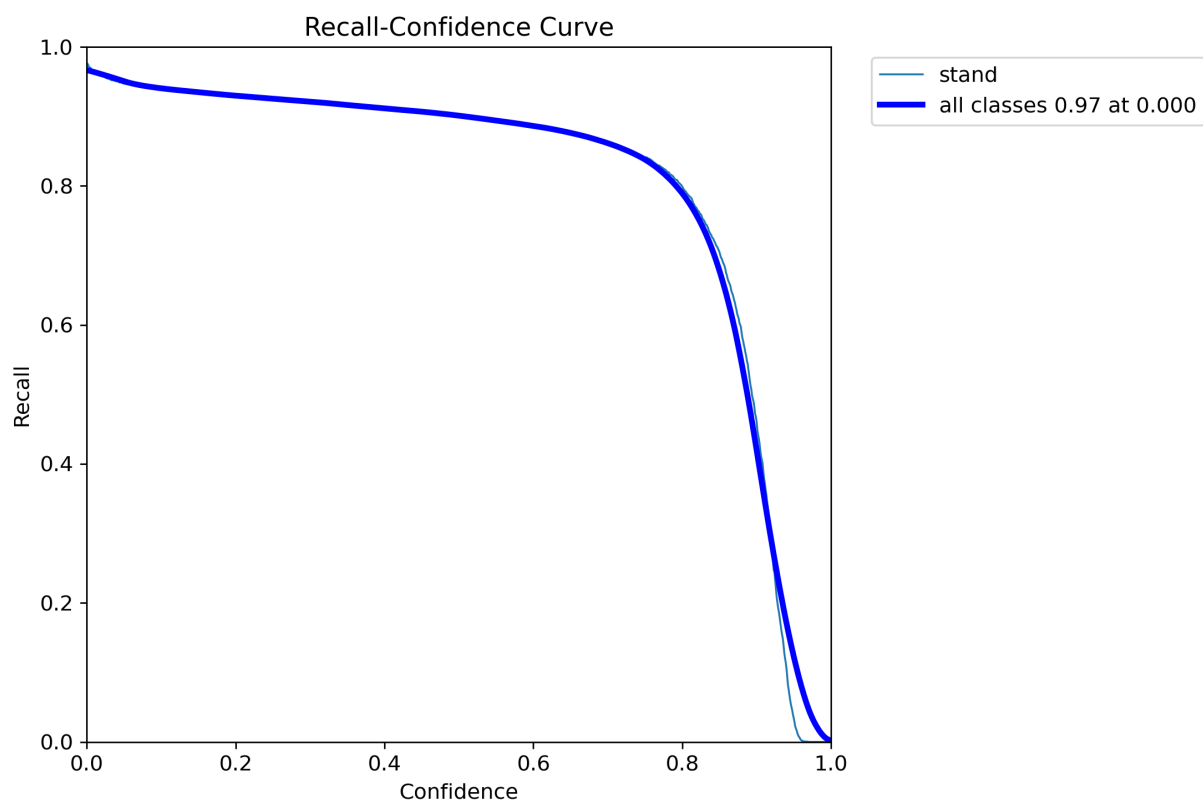
Hình 14: BoxF1_curve của talk_model



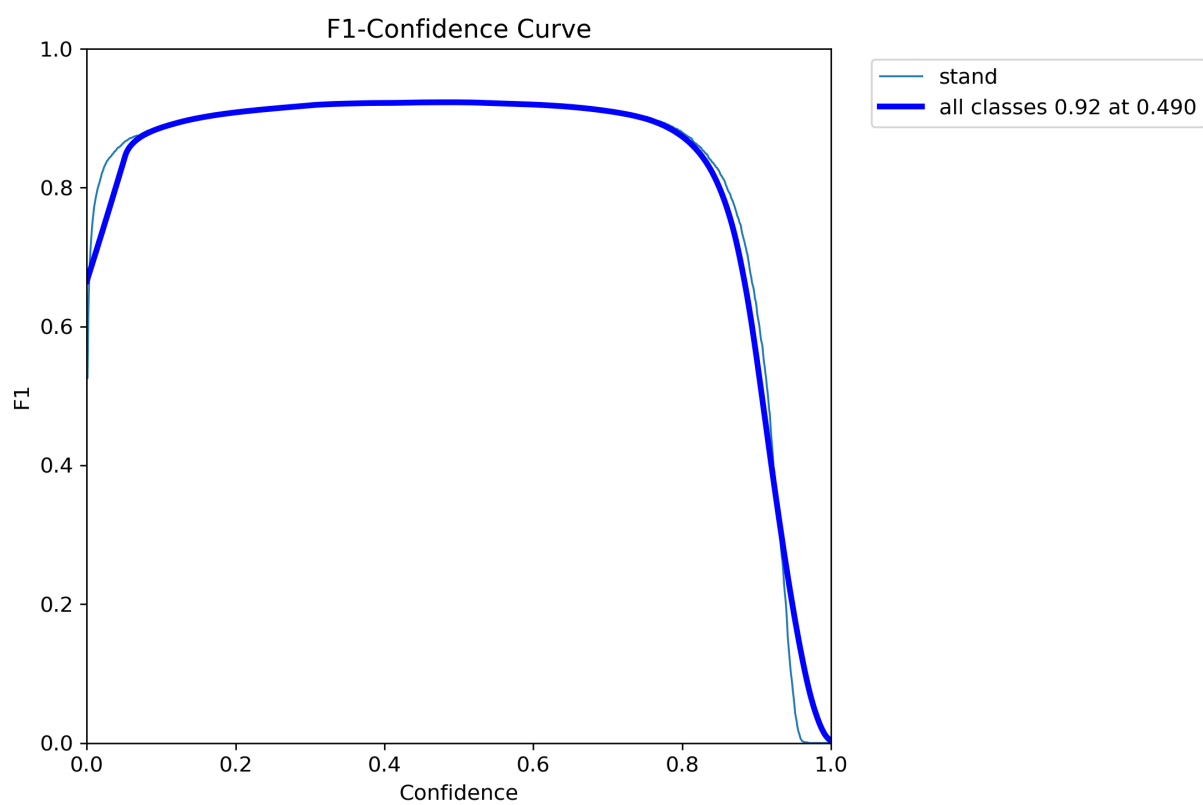
Hình 15: BoxPR_curve của talk_model



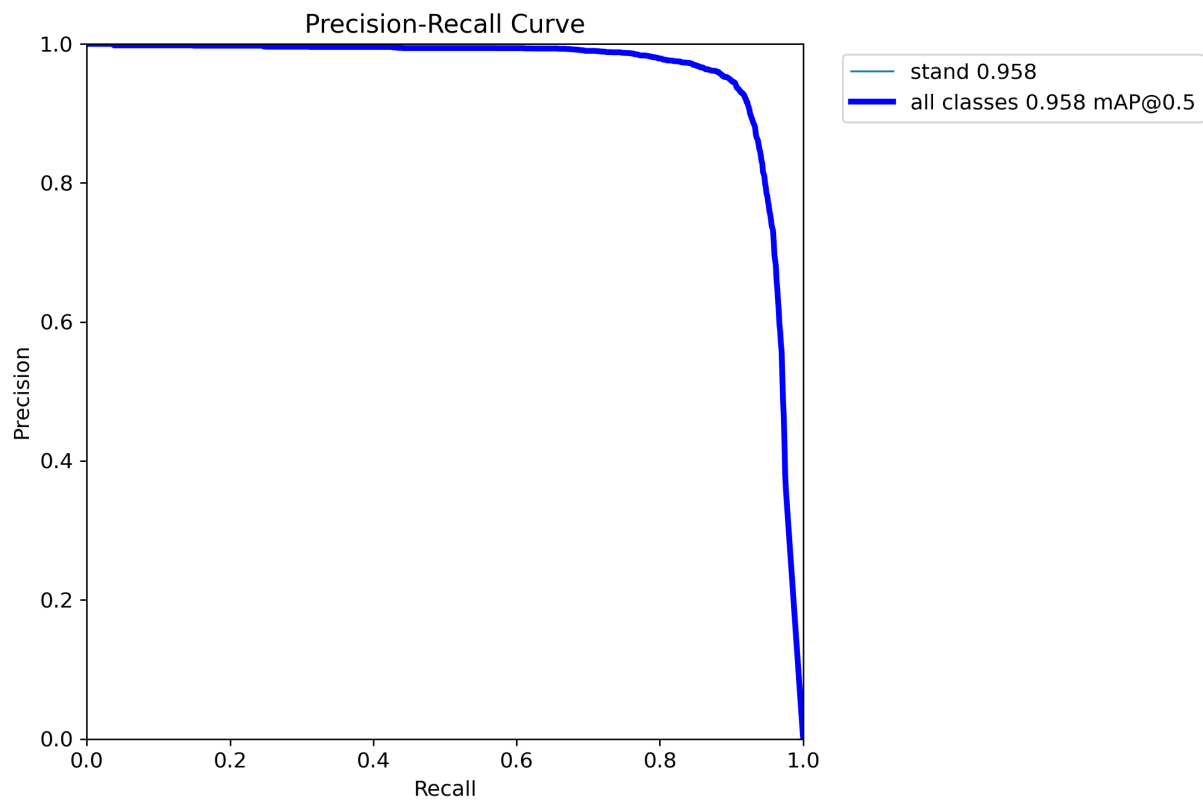
Hình 16: BoxP_curve của stand_model



Hình 17: BoxR_curve của stand_model



Hình 18: BoxF1_curve của stand_model



Hình 19: BoxPR_curve của stand_model

5.2.2 Confusion matrices

Confusion matrices cho từng model thể hiện khả năng phân loại của model trên validation set.

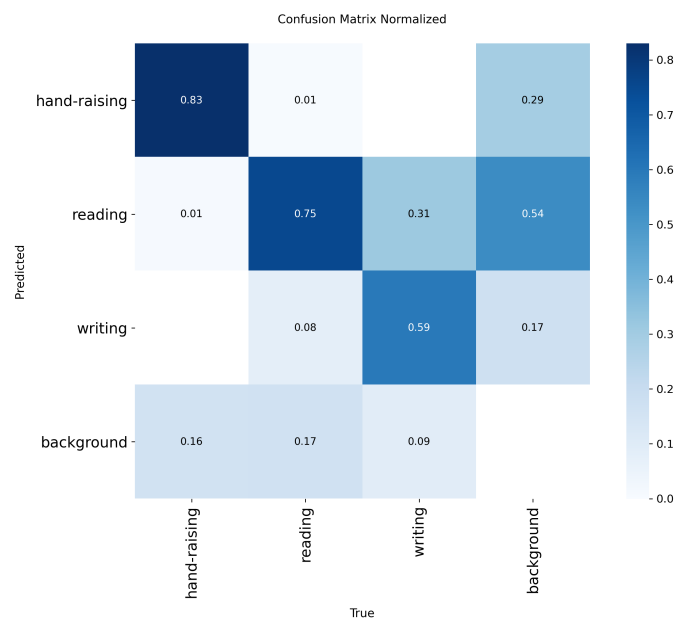
Confusion matrix cho thấy:

- Độ chính xác phân loại cho từng class
- Các trường hợp nhầm lẫn giữa các class
- Tổng số predictions đúng/sai cho mỗi class

hand-raise_read_write model

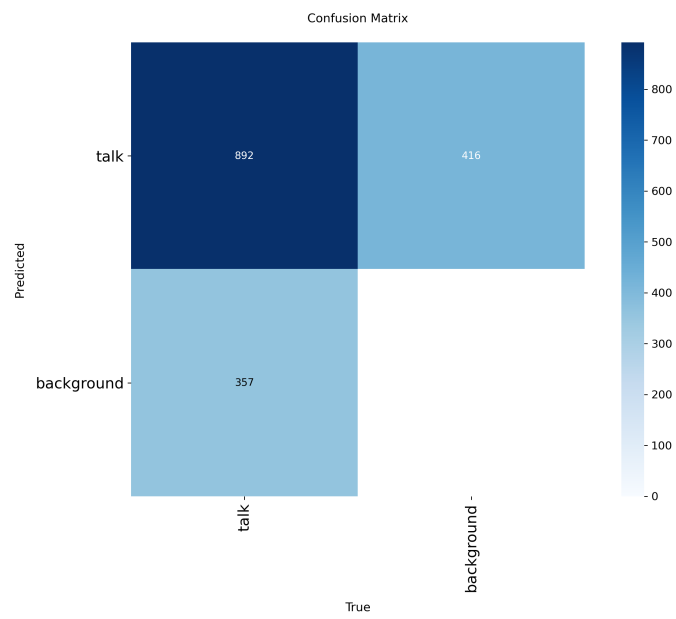


Hình 20: Confusion matrix của hand-raise_read_write model

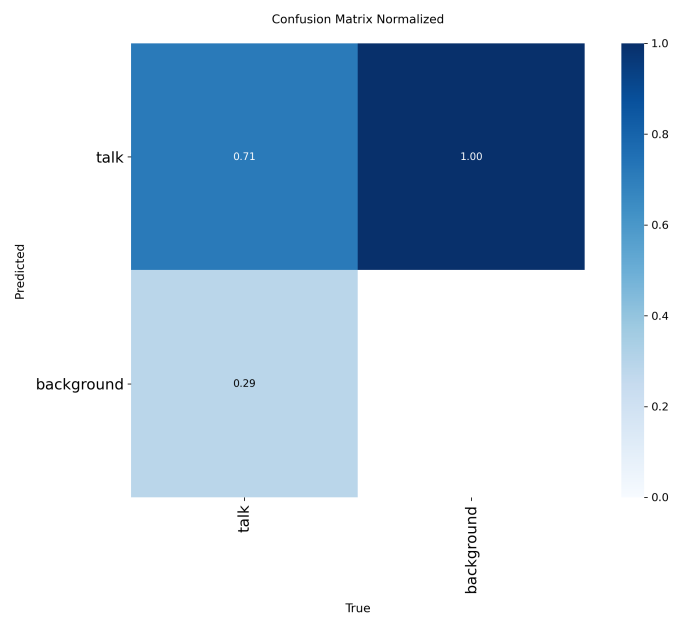


Hình 21: Confusion matrix normalized của hand-raise_read_write model

talk_model

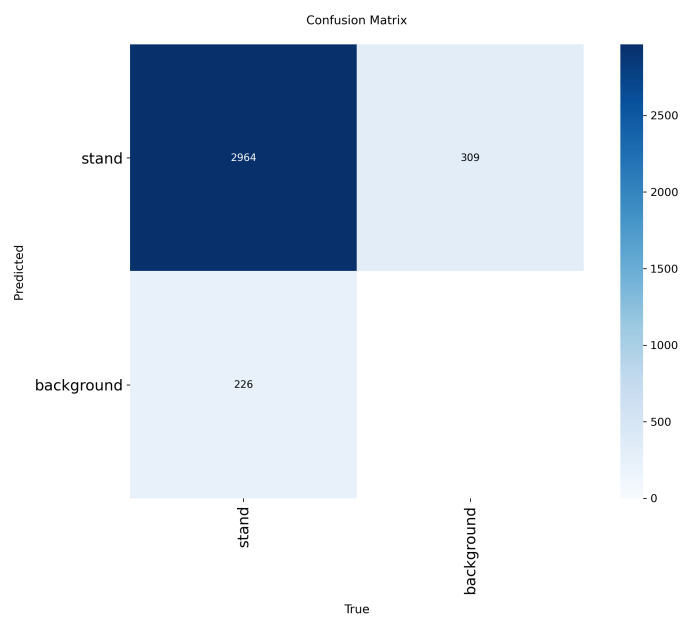


Hình 22: Confusion matrix của talk_model

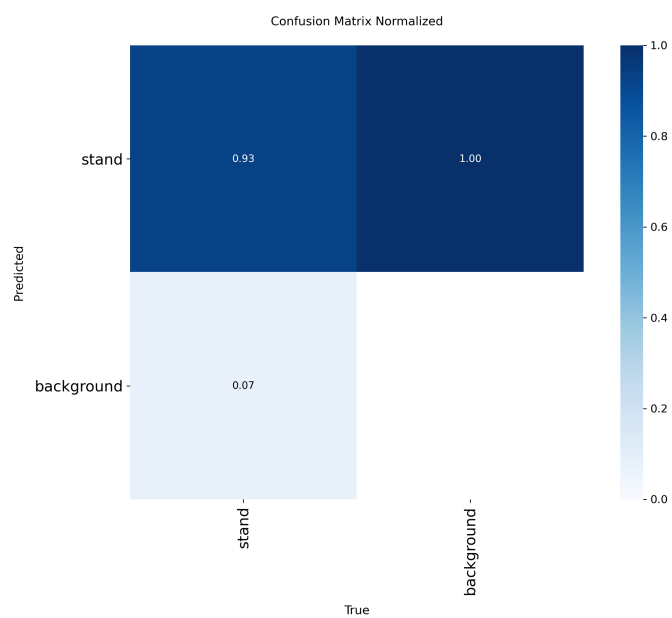


Hình 23: Confusion matrix normalized của talk_model

stand_model



Hình 24: Confusion matrix của stand_model



Hình 25: Confusion matrix normalized của stand_model

5.2.3 Kết quả đánh giá chi tiết

Bảng tổng hợp các metrics cuối cùng sau khi training hoàn tất

Model	<i>Precision</i> (%)	<i>Recall</i> (%)	<i>F1</i> (%)	<i>mAP@0.5</i> (%)	<i>mAP@0.5 : 0.95</i> (%)	Training time (h)	Final epoch
hand-raise_read_write	70.14	71.09	70.61	75.68	59.21	3.03	100
talk	81.50	64.93	72.24	74.44	57.79	3.30	100
stand	94.69	89.93	92.24	95.28	79.36	4.90	90

5.2.4 So sánh giữa các models và repo trước

Bảng so sánh tổng hợp

Metrics	hand-raise_read_write	talk	stand	Repo Classroom-Behavior-Detector
<i>Precision</i> (%)	70.14	81.50	94.69	71.55
<i>Recall</i> (%)	71.09	64.93	89.93	73.30
<i>F1</i> (%)	70.61	72.24	92.24	72.41
<i>mAP@0.5</i> (%)	75.68	74.44	95.28	75.46
<i>mAP@0.5 : 0.95</i> (%)	59.21	57.79	79.36	60.68
Training time (h)	3.03	1.30	4.90	N/A
Architecture	YOL012s	YOL012s	YOL012s	YOL08x

Nhận xét

1. hand-raise_read_write_model có kết quả tương đương với model gốc:
 - Với 3 classes, model này đạt *Precision* và *Recall* tương đối cân bằng (khoảng 70%), tương đương với YOL08x (71.55% precision, 73.30% *Recall*)
 - *mAP@0.5* là 75.68%, tương đương với YOL08x (75.46%), nhưng sử dụng model nhẹ hơn (YOL012s vs YOL08x)
2. stand_model đạt kết quả tốt nhất:
 - Với *Precision* 94.69%, *Recall* 89.93%, và *mAP@0.5* là 95.28%, stand_model cho thấy khả năng phát hiện hành vi đứng rất tốt, vượt trội so với model gốc (YOL08x: 75.46% *mAP@0.5*). Điều này có thể do hành vi đứng có đặc trưng hình ảnh rõ ràng và dễ phân biệt, và việc sử dụng model chuyên biệt cho từng hành vi giúp tối ưu hóa tốt hơn
3. talk_model có *Precision* cao nhưng *Recall* thấp hơn:
 - *Precision* 81.50% cho thấy model phát hiện talk khá chính xác, cao hơn đáng kể so với model gốc (71.55%)
 - *Recall* 64.93% cho thấy model có thể bỏ sót một số trường hợp nói chuyện. Điều này có thể do hành vi talk có nhiều biến thể và góc nhìn khác nhau

4. $mAP@0.5 : 0.95$:

- `stand_model` vượt trội với 79.36%, cao hơn đáng kể so với model gốc (60.68%), cho thấy khả năng localization tốt ở nhiều mức IoU khác nhau
- `hand-raise_read_write_model` và `talk_model` có $mAP@0.5 : 0.95$ thấp hơn một chút so với model gốc (59.21%, 57.79% vs 60.68%), điều này có thể do độ phức tạp của các hành vi và sự đa dạng trong dataset

Ưu điểm của hệ thống mới

So với repo gốc sử dụng 1 model YOLO8x cho 3 hành vi (`hand-raise`, `read`, `writie`), hệ thống mới sử dụng 3 models chuyên biệt (`hand-raise_read_write`, `talk`, `stand`) với YOLO12s cho phép:

- `stand_model` đạt độ chính xác cao hơn đáng kể (94.69% *Precision*, 95.28% $mAP@0.5$ vs 71.55% *Precision*, 75.46% $mAP@0.5$ của model gốc)
- `talk_model` có *Precision* tốt hơn (81.50% vs 71.55%)
- Sử dụng YOLO12s (nhẹ hơn) nhưng vẫn đạt kết quả tương đương với YOLO8x (lớn hơn) cho `hand-raise_read_write`
- Khả năng mở rộng tốt hơn: dễ thêm/bớt models cho các hành vi mới
- Tối ưu hóa riêng cho từng hành vi thay vì một model chung
- Phát hiện được 5 hành vi (`hand-raise`, `read`, `write`, `talk`, `stand`) thay vì chỉ 3 hành vi như model gốc
- Tốc độ inference nhanh hơn do sử dụng YOLO12s (nhẹ hơn YOLO8x) và có thể chạy song song với threading

5.3 Cân bằng giữa độ chính xác và tốc độ

Phân tích sự đánh đổi giữa độ chính xác và tốc độ trong việc xử lý hình ảnh.

Model image size

1. $MODEL_IMG_SIZE = 640$:

- Accuracy: Cao nhất (mAP cao hơn)
- Speed: Chậm nhất (khoảng 5-8 FPS trên CPU)
- Memory: Cao nhất

2. $MODEL_IMG_SIZE = 416$:

- Accuracy: Tốt (giảm nhẹ so với 640, khoảng 2-3)
- Speed: Nhanh hơn đáng kể (khoảng 8-12 FPS trên CPU, tăng khoảng 50-60)
- Memory: Trung bình
- Cân bằng: Tốt nhất cho ứng dụng thực tế

3. $MODEL_IMG_SIZE = 320$:

- Accuracy: Giảm rõ rệt (khoảng 5-8% mAP)
- Speed: Rất nhanh (khoảng 12-18 FPS trên CPU)
- Memory: Thấp nhất
- Cân bằng: Phù hợp khi cần tốc độ cao, chấp nhận giảm độ chính xác

Frame skip

1. $FRAME_SKIP = 1$:

- Accuracy: Cao nhất (không bỏ sót frame nào)
- Speed: Chậm nhất (phải xử lý tất cả frames)
- Cân bằng: Không phù hợp cho video dài, tốn thời gian

2. $FRAME_SKIP = 5$:

- Accuracy: Tốt (vẫn bắt được hầu hết các hành vi)
- Speed: Nhanh hơn 5 lần so với $FRAME_SKIP = 1$
- Cân bằng: Cân bằng tốt, phù hợp cho video processing

3. $FRAME_SKIP = 10$:

- Accuracy: Giảm nhẹ (có thể bỏ sót một số hành vi ngắn)
- Speed: Rất nhanh
- Cân bằng: Phù hợp cho live camera

Max detections

1. $MAX_DETECTIONS = 6$:

- Accuracy: Có thể bỏ sót khi có nhiều học sinh
- Speed: Nhanh hơn
- Memory: Thấp hơn

2. $MAX_DETECTIONS = 20$:

- Accuracy: Tốt hơn, phát hiện được nhiều học sinh hơn
- Speed: Chậm hơn một chút (khoảng 5-10%)
- Memory: Cao hơn một chút
- Cân bằng: Phù hợp cho lớp học có nhiều học sinh

Kết luận

Cấu hình được lựa chọn là tối ưu:

- $MODEL_IMG_SIZE = 416$ cân bằng tốt giữa accuracy và speed
- $FRAME_SKIP = 5$ cho video và $FRAME_SKIP_CAMERA = 10$ cho live đảm bảo khả năng đánh giá trong thời gian thực
- $MAX_DETECTIONS = 20$ phù hợp cho môi trường thực tế
- $USE_THREADING = True$ tận dụng đa luồng để tăng tốc độ

Cấu hình này cho phép hệ thống đạt được độ chính xác tốt, trong khi vẫn đảm bảo tốc độ xử lý đủ nhanh cho ứng dụng thực tế.

6 Hệ thống ứng dụng và demonstration

6.1 Kiến trúc ứng dụng

Ứng dụng được tổ chức theo các modules, tách biệt rõ ràng giữa các thành phần. Cấu trúc thư mục như sau:

```
student-behavior-analysis/
├── modules/
│   ├── core/
│   │   ├── config.py          # Cấu hình
│   │   ├── model_loader.py    # Load models
│   │   ├── detection_utils.py # Xử lý detection, NMS, vẽ kết quả
│   │   ├── image_processor.py # Xử lý ảnh tĩnh
│   │   └── video_processor.py # Xử lý video và camera
│   ├── apps/
│   │   ├── app.py             # Gradio Web UI
│   │   └── live_camera_demo.py # OpenCV Live Camera Demo
│   └── models/
│       └── student_behavior/
│           ├── hand-raise_read_write_model/
│           ├── talk_model/
│           └── stand_model/
├── run_app.py                 # Entry point cho Gradio UI
├── run_live_camera.py         # Entry point cho Live Camera
├── requirements.txt           # Dependencies
└── README.md                  # File này
```

6.1.1 Core Modules

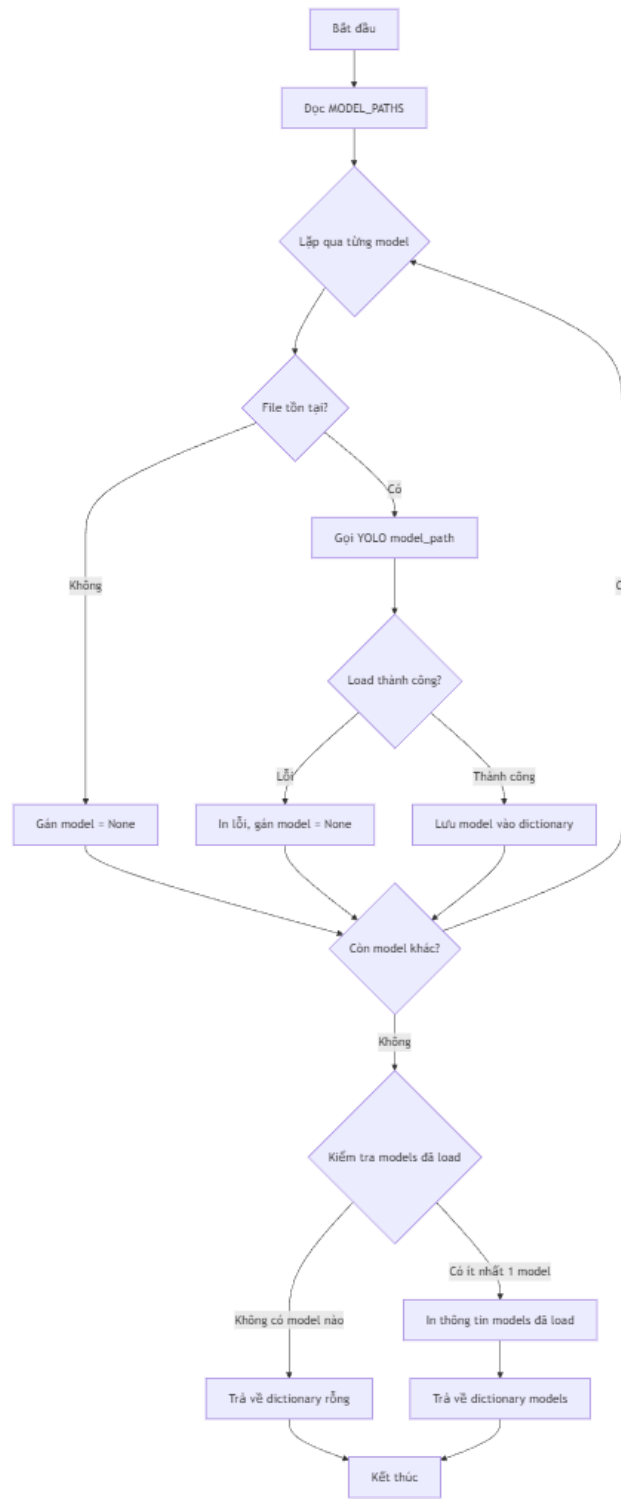
Chứa logic nghiệp vụ, không phụ thuộc vào giao diện người dùng.

config.py - Module cấu hình

- Camera: Độ phân giải 640x480
- Frame Processing:
 - + $FRAME_SKIP = 5$
 - + $FRAME_SKIP_CAMERA = 10$
- Model: Kích thước ảnh đầu vào là 416px
- Detection:
 - + Confidence threshold: 0.5
 - + IOU threshold: 0.45 (cho NMS)
 - + Max detections: 20
- Threading: $USE_THREADING = True$ (xử lý song song với queue $maxsize = 5$)
- Màu sắc: $CLASS_COLOR_MAP$ cho từng hành vi
- Video: Codec H.264 (avc1)

model_loader.py - Module tải models

- Định nghĩa đường dẫn cho 3 models hand-raise_read_write, talk, stand
- load_all_models(): Tải tất cả models song song
 1. Kiểm tra file tồn tại, bắt exception khi tải
 2. Trả về dictionary {'model_name': model_object}



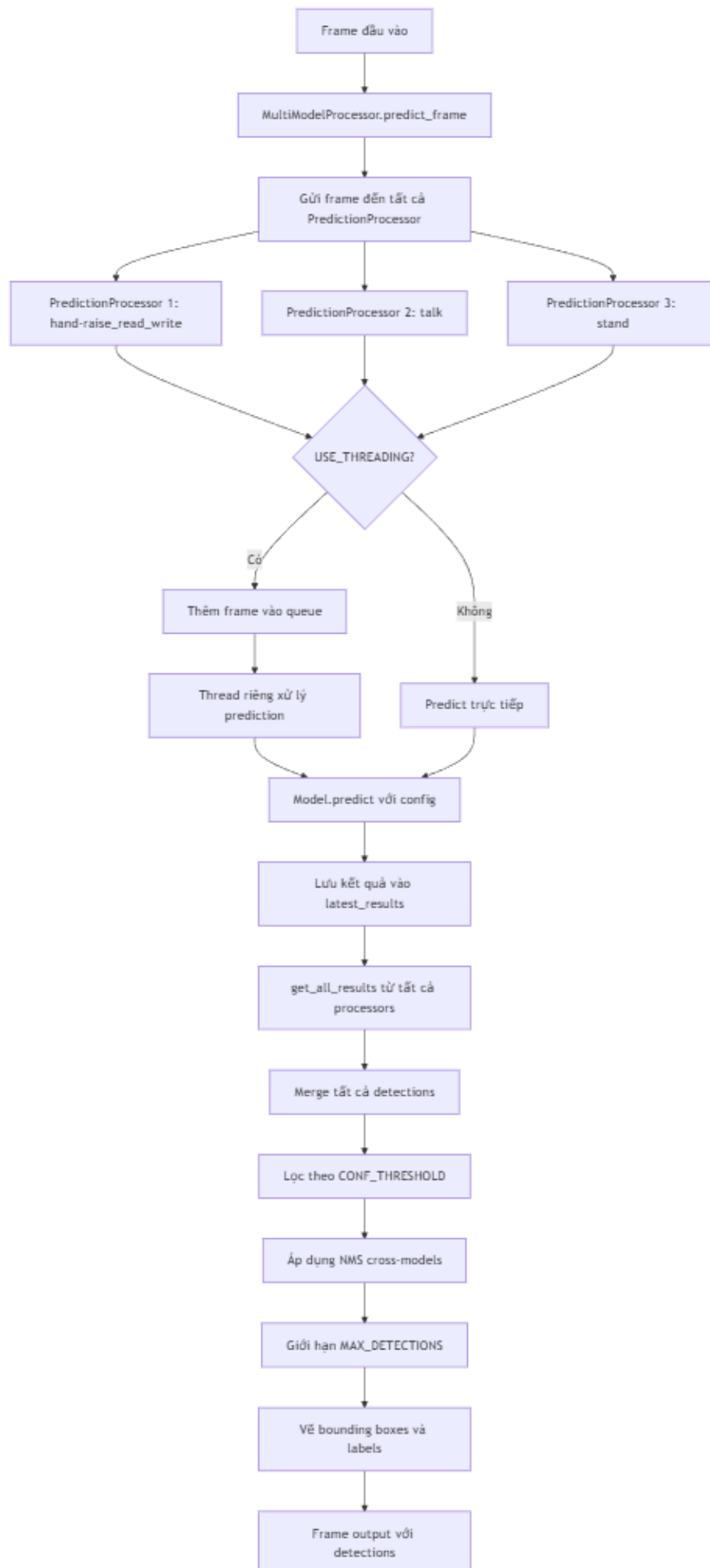
`detection_utils.py` - Module xử lý detection

1. Các hàm chính:

- `calculate_iou()`: Tính Intersection over Union giữa 2 bounding boxes
- `apply_nms_cross_models()`: Áp dụng Non-Maximum Suppression giữa các models để loại bỏ detections trùng lặp
- `merge_and_filter_results()`: Gộp kết quả từ nhiều models, lọc theo confidence threshold
- `draw_detections()`: Vẽ bounding boxes và labels lên frame
- `draw_multiple_detections()`: Vẽ detections từ nhiều models

2. Các class:

- `PredictionProcessor`: Xử lý prediction cho 1 model, hỗ trợ threading
 - + Sử dụng queue (*maxsize* = 5) để tránh block main thread
 - + Thread riêng xử lý prediction song song
 - + Hỗ trợ `predict_frame()` với tham số wait (đợi queue xử lý xong với *timeout* = 0.1s)
 - + Lưu kết quả mới nhất trong `latest_results` với thread-safe lock
- `MultiModelProcessor`: Quản lý nhiều `PredictionProcessor`
 - + Chạy prediction song song cho tất cả models
 - + Gộp kết quả từ tất cả models
 - + Hỗ trợ `predict_frame()` với wait để đảm bảo prediction được cập nhật

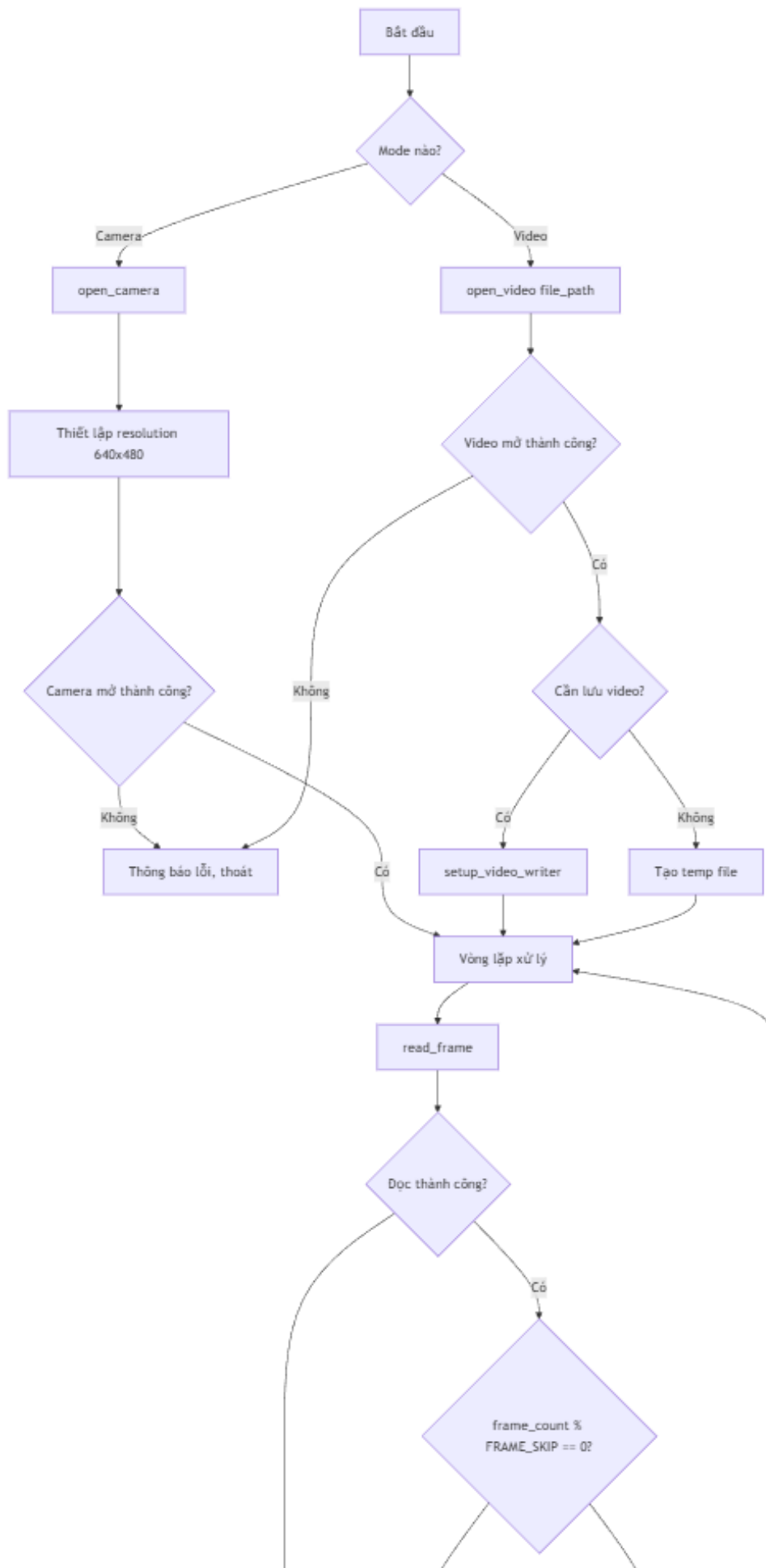


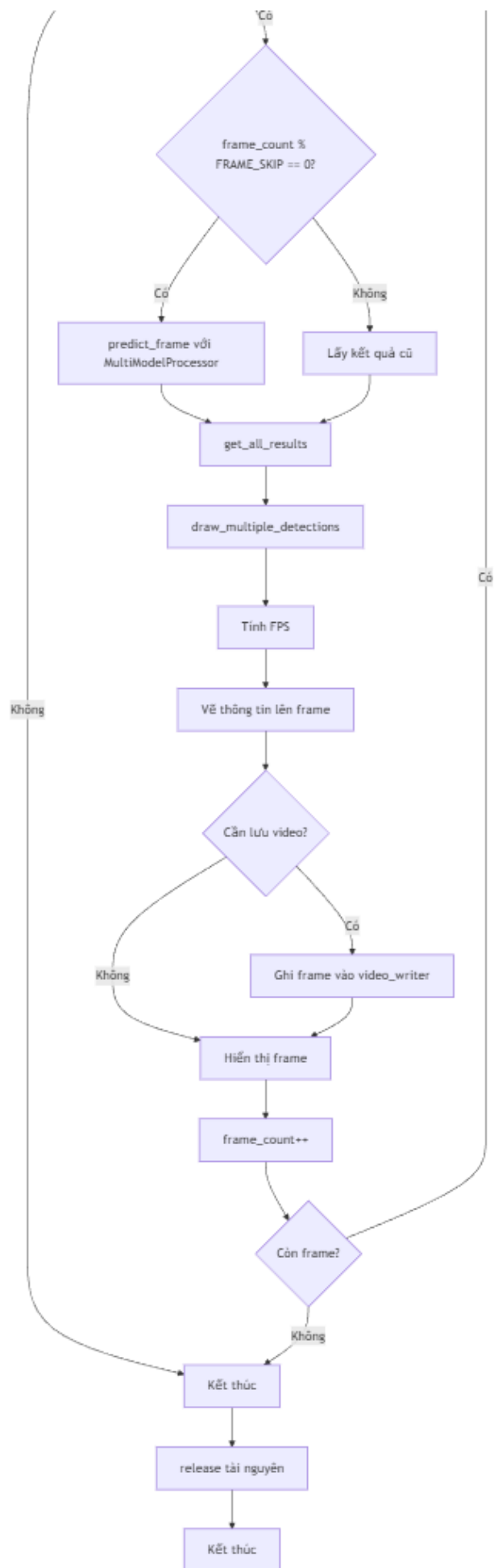
`image_processor.py` - Module xử lý ảnh

- `process_image()`: Xử lý ảnh tĩnh
 1. Đọc ảnh từ file path
 2. Resize nếu quá lớn ($MAX_DISPLAY_SIZE = 1200px$)
 3. Gọi `MultiModelProcessor` để predict
 4. Vẽ detections và trả về kết quả
- `show_image()`: Hiển thị ảnh với OpenCV (dùng trong live camera demo)

`video_processor.py` - Module xử lý video và camera

- `open_camera()`: Mở camera với cấu hình từ config
- `open_video()`: Mở video file
- `read_frame()`: Đọc frame từ camera/video
- `process_frame()`: Xử lý một frame
 1. Gọi prediction mỗi *FRAME_SKIP* hoặc *FRAME_SKIP_CAMERA* tùy mode
 2. Vẽ detections
 3. Tính FPS
 4. Hiển thị thông tin
 5. Ghi vào video nếu đang lưu
- `setup_video_writer()`: Thiết lập `VideoWriter` để lưu video
- `release()`: Giải phóng tài nguyên





6.1.2 Application Modules

Cung cấp giao diện người dùng, sử dụng core modules để xử lý.

app.py - Gradio Web UI

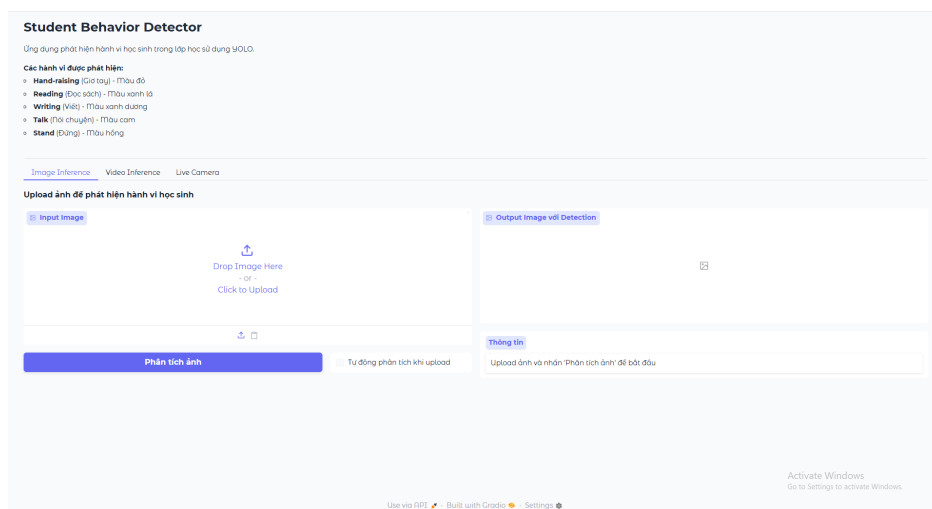
Module tạo giao diện web với Gradio, tích hợp 3 chức năng.

1. Cấu trúc:

- Khởi tạo: Load tất cả models khi khởi động, tạo `MultiModelProcessor` global
- 3 Tabs chính:
 - + Image Inference: Upload và phân tích ảnh
 - + Video Inference: Upload và phân tích video, có option lưu
 - + Live Camera: Khởi động OpenCV demo trong cửa sổ riêng

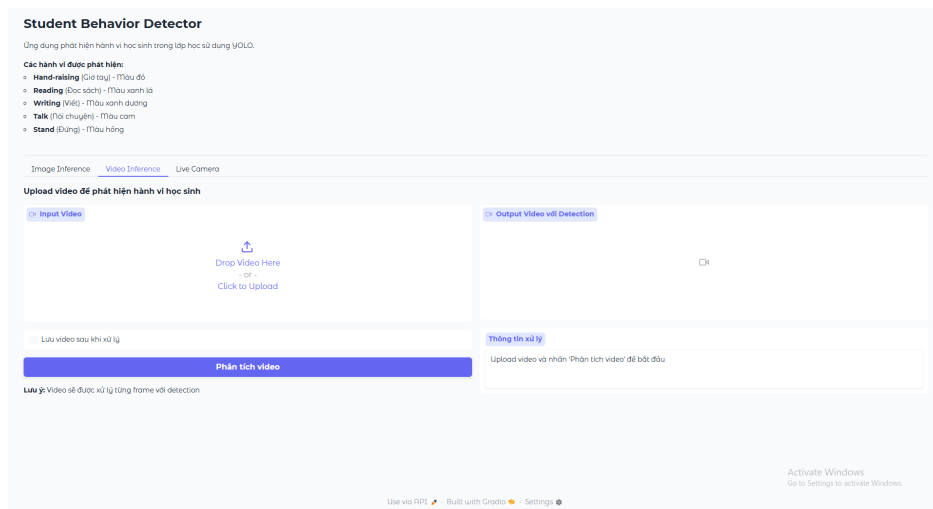
2. Các hàm xử lý:

- `process_image_ui()`: Xử lý ảnh cho Gradio UI
 - + Nhận file path hoặc numpy array
 - + Chuyển đổi RGB/BGR
 - + Gọi `process_image()` từ `image_processor`

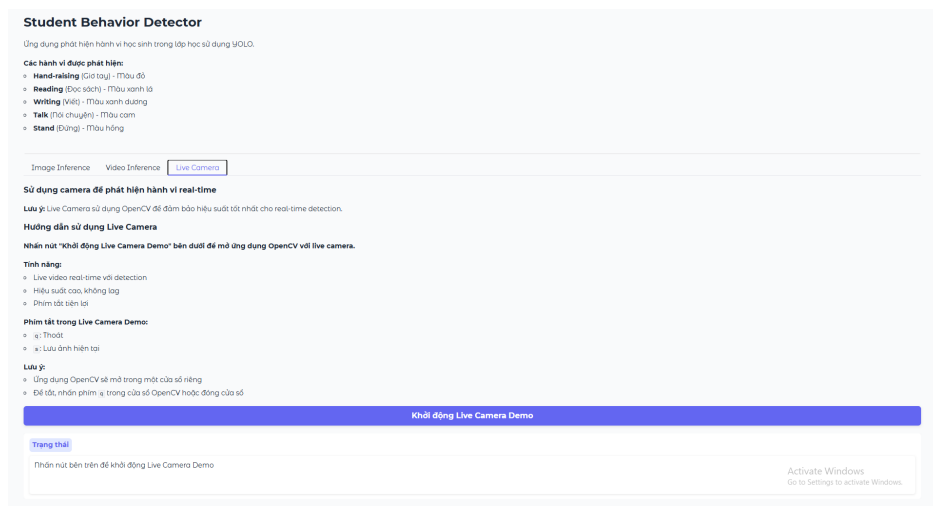


- `process_video_ui()`: Xử lý video cho Gradio UI
 - + Tạo `VideoProcessor` với `MultiModelProcessor`
 - + Đọc từng frame từ video
 - + Mỗi `FRAME_SKIP` frame: gọi `predict_frame()` với `wait = True` để đảm bảo prediction được cập nhật
 - + Vẽ detections lên frame

- + Ghi frame vào video output
- + Hiện thị progress mỗi 30 frame
- + Trả về video đã xử lý và thông tin



- `launch_opencv_demo()`: Khởi động Live Camera Demo
 - + Tìm file `run_live_camera.py`
 - + Chạy bằng lệnh Windows `start` trong cửa sổ mới

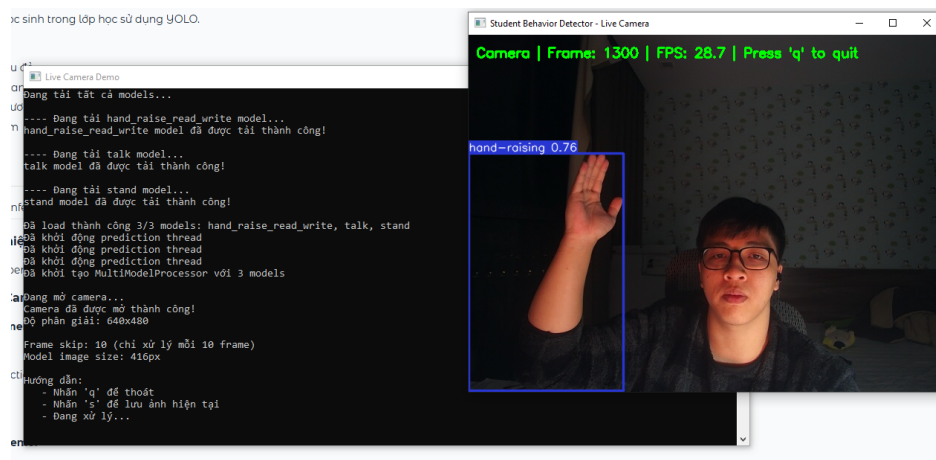


live_camera_demo.py - OpenCV Live Camera Demo

Ứng dụng standalone dùng OpenCV cho live camera

1. Load models khi khởi động
2. Mở camera và hiển thị real-time
3. Xử lý frame với VideoProcessor
4. Phím tắt:

- q: Thoát
- s: Lưu ảnh hiện tại

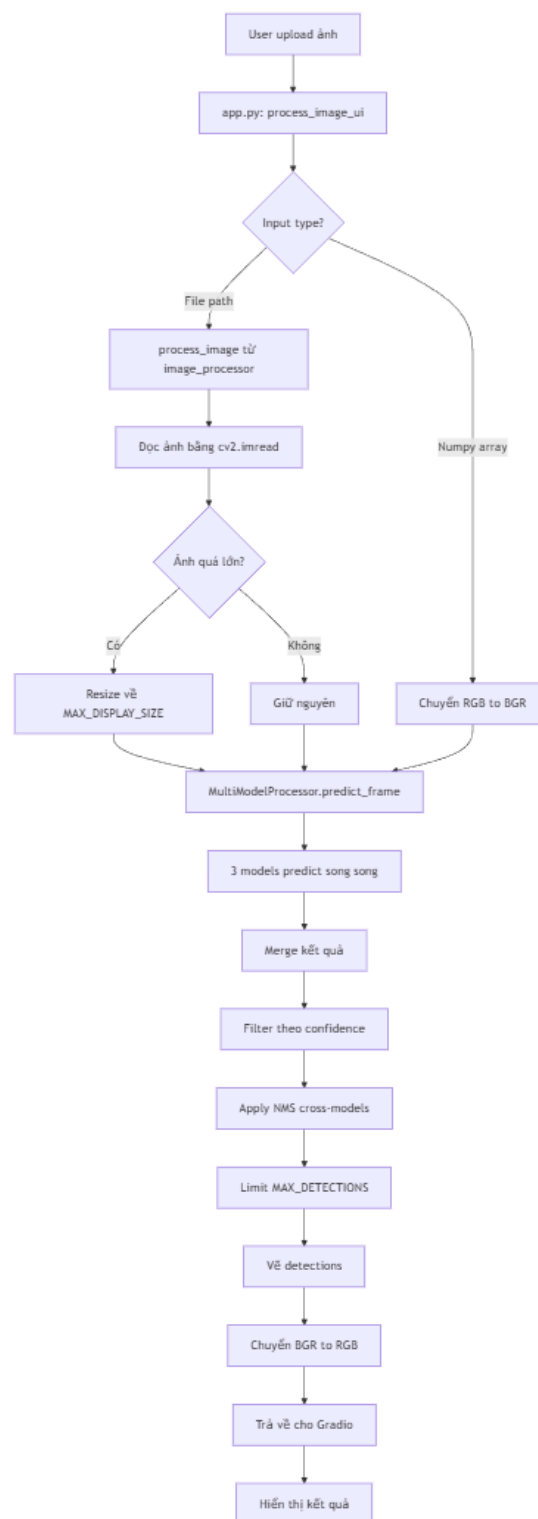


6.1.3 Entry Points

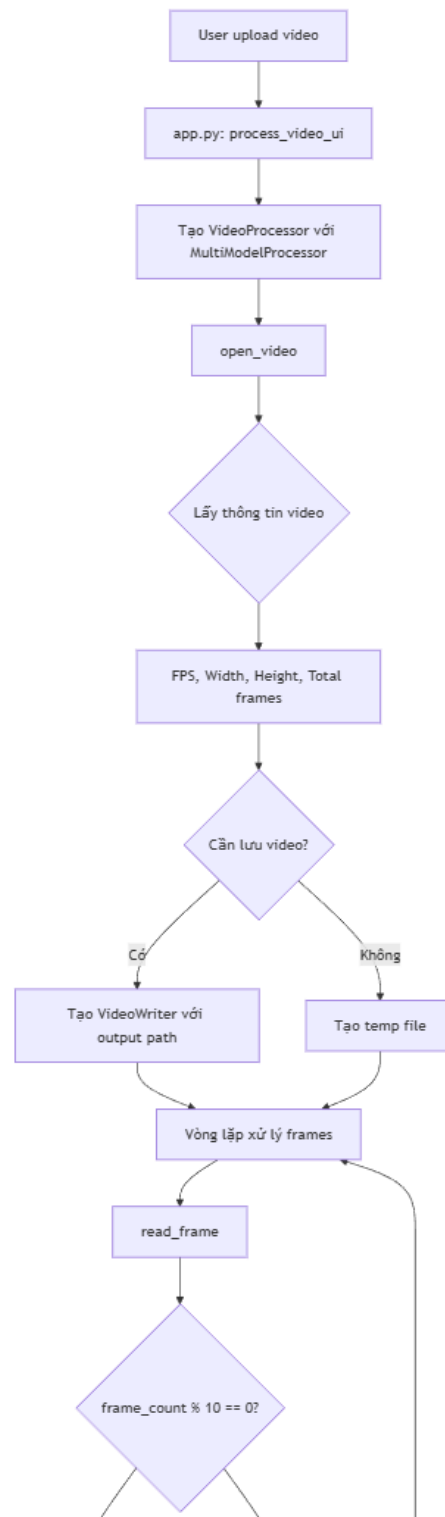
- run_app.py: Khởi động Gradio Web UI
 1. Setup Python path
 2. Import và launch Gradio demo
 3. Server chạy tại <http://localhost:7860>
- run_live_camera.py: Khởi động Live Camera Demo
 1. Setup Python path
 2. Gọi hàm main() từ live_camera_demo
 3. Có thể được khởi chạy trực tiếp hoặc truy cập tab Live Camera khi khởi chạy run_app.py

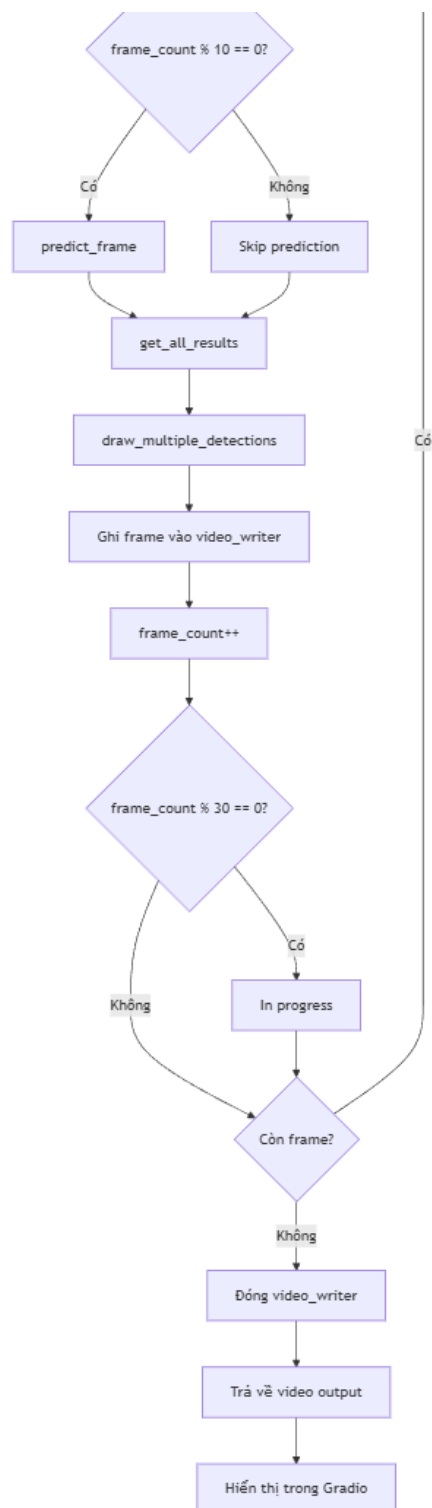
6.1.4 Luồng xử lý tổng thể

1. Luồng xử lý ảnh

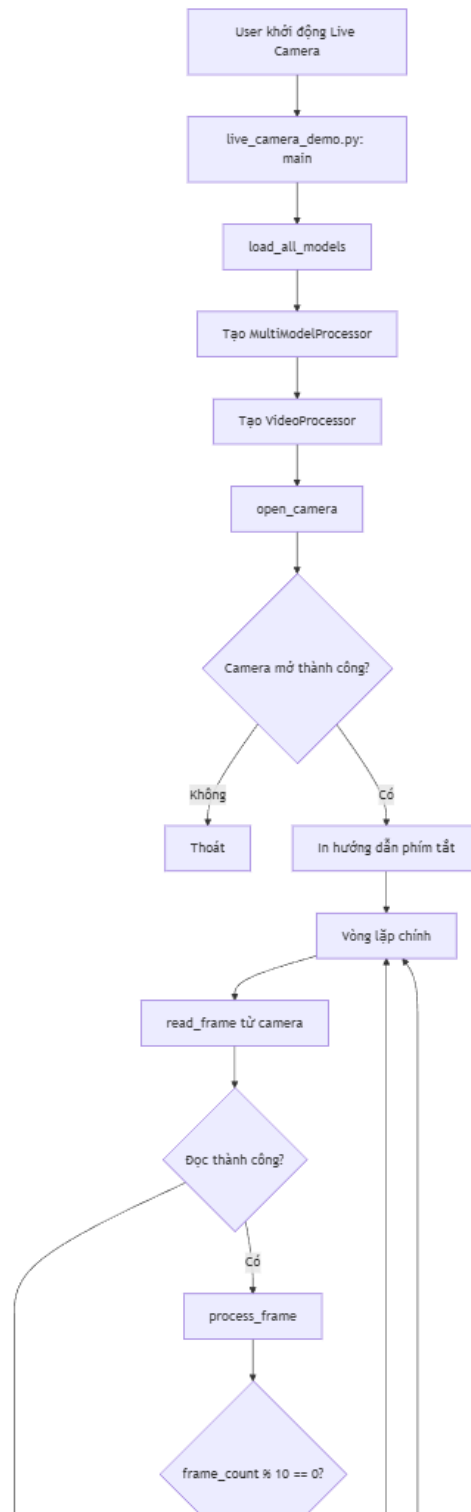


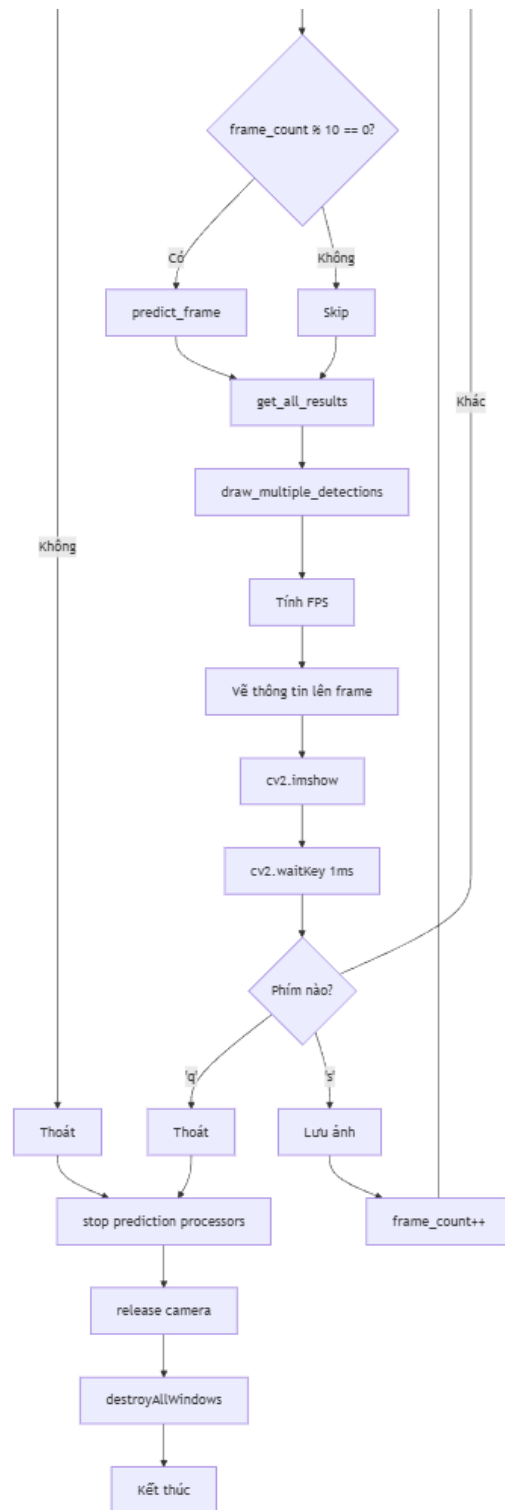
2. Luồng xử lý video





3. Luồng xử lý camera





6.1.5 Điểm mạnh của kiến trúc

Cấu trúc dự án được lựa chọn theo kiến trúc này nhằm đảm bảo hệ thống hoạt động ổn định, dễ mở rộng và bảo trì:

- Core logic độc lập với UI, dễ test và bảo trì
- Multi-model support để thêm/bớt models, chỉ cần cập nhật *MODEL_PATHS*
- Threading tối ưu giúp Prediction không block main thread, queue *maxsize* = 5 để cân bằng tốc độ và bộ nhớ
- Cross-model NMS loại bỏ detections trùng lặp giữa các models, tăng độ chính xác
- Tái sử dụng core modules cho nhiều ứng dụng (Web UI, Live Camera, có thể mở rộng thêm)
- Mỗi module có trách nhiệm rõ ràng, dễ bảo trì và cải thiện
- Cấu hình tập trung trong *config.py*, dễ điều chỉnh

6.2 Tính năng và cấu hình

6.2.1 Các tính năng chính

Image inference

1. Người dùng upload một ảnh lên Web UI
2. Ảnh được resize nếu quá lớn, sau đó được đưa qua 3 model YOLO chạy song song
3. Kết quả của các model được gộp lại, lọc theo ngưỡng confidence, áp dụng NMS giữa các model để loại bỏ bounding box trùng lặp, sau đó vẽ lại lên ảnh
4. Ảnh kết quả được hiển thị trực tiếp trên Gradio.

Video processing

1. Người dùng upload một file video
2. Hệ thống sử dụng VideoProcessor để đọc từng frame của video:
 - Không phải frame nào cũng được đưa vào model, mà chỉ xử lý mỗi *FRAME_SKIP* frame (mặc định là 10) để tối ưu hiệu năng
 - Mỗi frame được xử lý tương tự như ảnh tĩnh: multi-model inference + cross-model NMS + vẽ bounding boxes
3. Người dùng có thể chọn:
 - Chỉ xem video đã được xử lý (file tạm)
 - Lưu thành file video mới
4. Trong quá trình xử lý, hệ thống in ra số frame đã xử lý và tiến độ

Live camera detection

1. Chạy qua file `run_live_camera.py` hoặc nút Khởi động Live Camera Demo trong Web UI
2. Ứng dụng `live_camera_demo.py` sử dụng OpenCV để:
 - Mở webcam
 - Đọc frame liên tục
 - Áp dụng multi-model inference tương tự video
 - Hiển thị trực tiếp kết quả lên cửa sổ OpenCV (real-time)
3. Phím tắt:
 - q: thoát chương trình
 - s: lưu lại frame hiện tại thành file ảnh

6.2.2 Web UI với Gradio

Web UI được xây dựng bằng Gradio trong `modules/apps/app.py` với 3 tab chính:

Tab Image Inference

1. Image input: chọn/drag-drop ảnh
2. Nút “Phân tích ảnh” và tùy chọn “Tự động phân tích khi upload”
3. Image output: hiển thị ảnh đã vẽ bounding boxes
4. Textbox thông tin: báo trạng thái phân tích

Tab Video Inference

1. Video input: upload file
2. Checkbox “Lưu video sau khi xử lý”
3. Nút “Phân tích video”
4. Video output: phát video đã được xử lý
5. Textbox thông tin: số frame xử lý, đường dẫn file output
6. File download: tải video đã xử lý nếu người dùng chọn lưu

Tab Live Camera

1. Hiển thị hướng dẫn và nút “Khởi động Live Camera Demo”
2. Gọi `run_live_camera.py` trong một cửa sổ console mới, chạy ứng dụng OpenCV live camera

6.2.3 Config parameters

Toàn bộ tham số cấu hình tập trung trong `modules/core/config.py`, giúp dễ dàng tinh chỉnh

Camera & Video

- $CAMERA_WIDTH = 640$, $CAMERA_HEIGHT = 480$: độ phân giải camera
- $VIDEO_CODEC = avc1$: sử dụng H.264 cho video output (tương thích trình duyệt), có fallback sang mp4v ở code xử lý video

Model & Detection

- $MODEL_IMG_SIZE = 320$: kích thước ảnh đưa vào model YOLO resize về 320x320)
- $CONF_THRESHOLD = 0.5$: ngưỡng confidence; detection có $confidence < 0.5$ bị loại bỏ
- $IOU_THRESHOLD = 0.45$: ngưỡng IoU cho NMS; 2 boxes trùng nếu $IoU \geq 0.45$
- $MAX_DETECTIONS = 6$: giới hạn tối đa 6 detection/frame để giảm noise và tăng tốc.

Hiệu năng

- $FRAME_SKIP = 10$: chỉ chạy model mỗi 10 frame, các frame giữa dùng lại kết quả gần nhất
- $USE_THREADING = True$: sử dụng threading cho prediction, tránh block main loop đọc frame

Màu sắc class

- $CLASS_COLOR_MAP$
 1. hand-raise: đỏ
 2. read: xanh lá
 3. write: xanh dương
 4. talk: cam
 5. stand: hồng

6.3 Hướng dẫn sử dụng

6.3.1 Cài đặt và setup

Bước 1: Chuẩn bị môi trường

Đảm bảo máy có:

- Python 3.8+
- Webcam (nếu dùng live camera để thử nghiệm)

Bước 2: Clone hoặc tải mã nguồn

Clone repo từ GitHub, chạy các lệnh sau trong terminal:

```
git clone https://github.com/Tienchan27/student-behavior-analysis.git
```

```
cd student-behavior-analysis
```

Bước 3: Cài đặt dependencies

Cài các thư viện cần thiết:

```
pip install -r requirements.txt
```

6.3.2 Sử dụng Web UI

Bước 1: Chạy Web UI

```
python run_app.py
```

Sau khi chạy, terminal sẽ in:

```
Đ
"(*ang ởkhi động Gradio Web UI...*)"
"(*Truy ập cập ại*): (*\url{http://localhost:7860}*)"
```

Bước 2: Mở trình duyệt và truy cập <http://localhost:7860>

Tab Image Inference

1. Chọn tab Image Inference
2. Nhấn vào ô Input Image để upload ảnh, hoặc dán từ clipboard
3. Tick Tự động phân tích khi upload nếu muốn xử lý ngay sau khi upload
4. Hoặc:
 - Upload ảnh
 - Nhấn nút Phân tích ảnh
5. Kết quả:
 - Ảnh bên phải hiển thị bounding boxes màu sắc tương ứng từng hành vi
 - Ô Thông tin hiển thị trạng thái như:
 - + Upload ảnh và nhấn Phân tích ảnh để bắt đầu
 - + Đã tự động phân tích ảnh
 - + Đã phân tích xong ảnh

Tab Video Inference

1. Chọn tab Video Inference
2. Upload video ở ô Input Video
3. Nếu muốn lưu file video đã xử lý: Tick Lưu video sau khi xử lý
4. Nhấn nút Phân tích video

5. Hệ thống sẽ:

- Đọc video, xử lý từng frame (mỗi 10 frame chạy model)
- Hiển thị tiến độ trong terminal (số frame và phần trăm)

6. Kết quả:

- Output Video hiển thị video đã xử lý (có detections)
- Thông tin xử lý ghi lại:
 - + Số frame đã xử lý
 - + Đường dẫn video output (file thật hoặc file tạm)

7. Nếu chọn lưu video: Nút Download Video hiển thị, click để tải.

Tab Live Camera

1. Chọn tab Live Camera.

2. Đọc hướng dẫn nhanh:

- Cách khởi động
- Phím tắt q, s

3. Nhấn nút Khởi động Live Camera Demo

4. Một cửa sổ console mới và cửa sổ OpenCV sẽ xuất hiện:

- Cửa sổ OpenCV hiển thị camera với bounding boxes real-time
- Thông tin hướng dẫn cũng được in trong console:
 - + Nhấn q để thoát
 - + Nhấn s để lưu ảnh hiện tại

5. Nhấn q trong cửa sổ OpenCV để đóng.

6.3.3 Live Camera Demo

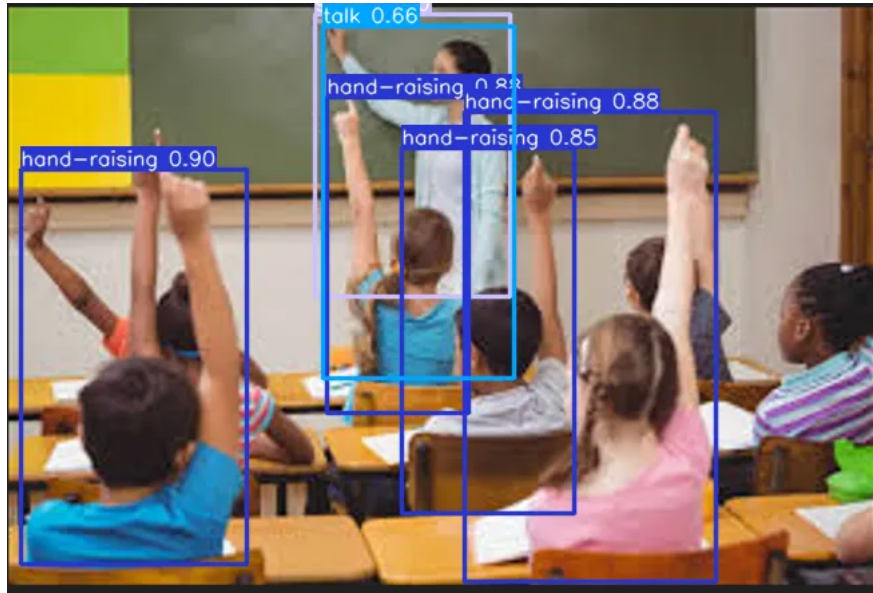
Ngoài Web UI, có thể chạy live camera trực tiếp bằng terminal:

```
python run\_live\_camera.py
```

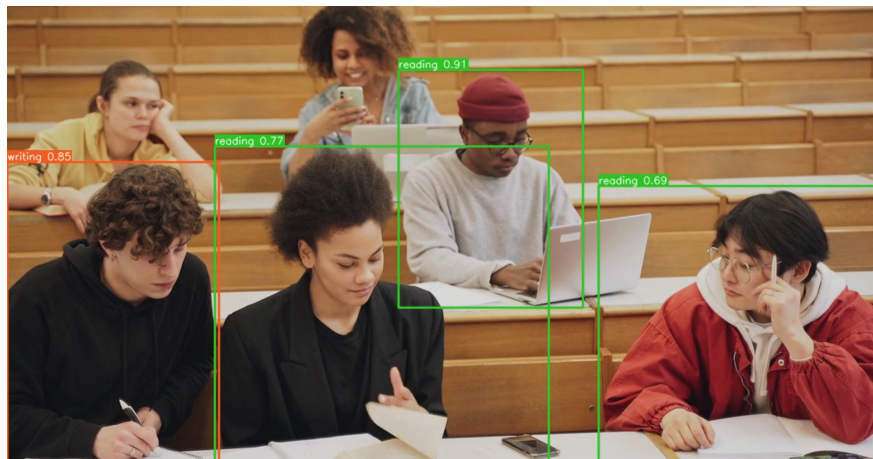
Sau đó, dùng tương tự như sau khi khởi động qua tab Live Camera ở Web UI

6.3.4 Demo scenarios và sample outputs

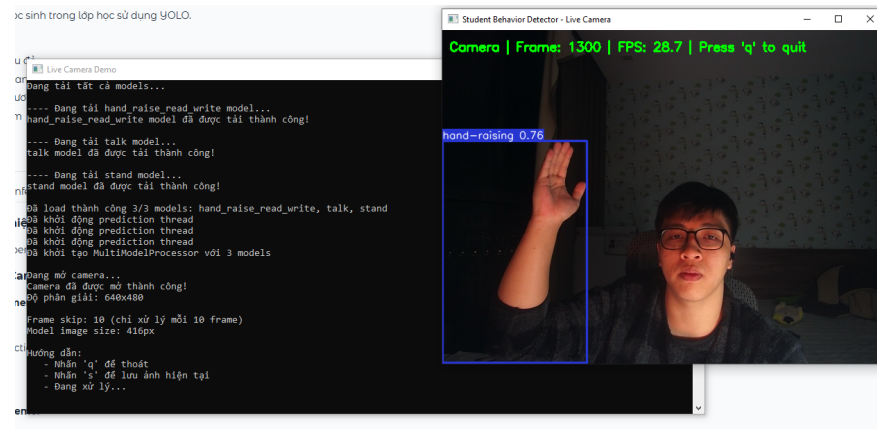
1. Demo 1 – Ảnh lớp học tĩnh:



2. Demo 2 – Video tiết học:



3. Demo 3 – Live camera:



7 Kết luận

7.1 Hạn chế của hệ thống

- Độ chính xác của mô hình còn bị ảnh hưởng trong các tình huống che khuất, ánh sáng kém hoặc góc quay không thuận lợi
- Một số hành vi có sự tương đồng cao (ví dụ *Read* và *Write*) khiến mô hình dễ nhầm lẫn
- Hệ thống chủ yếu dựa trên dữ liệu hình ảnh, chưa khai thác đầy đủ thông tin thời gian dài hạn trong video
- Quy mô dữ liệu còn hạn chế và chưa đa dạng, ảnh hưởng đến khả năng tổng quát hóa của mô hình

7.2 Hướng phát triển trong tương lai

- Sử dụng các mô hình Temporal Learning (LSTM, Temporal CNN, Transformer) để khai thác tốt hơn thông tin chuỗi thời gian
- Mở rộng tập dữ liệu với nhiều lớp học, góc quay và điều kiện môi trường khác nhau
- Nâng cấp hệ thống để hỗ trợ nhận diện nhiều học sinh đồng thời và theo dõi hành vi theo thời gian thực
- Kết hợp thêm các nguồn dữ liệu khác như âm thanh hoặc pose estimation để nâng cao độ chính xác
- Ứng dụng hệ thống vào các nền tảng smart classroom hoặc công cụ hỗ trợ giảng dạy thông minh

Tài liệu tham khảo

- Abadi, M., Chu, A., Goodfellow, I., McMahan, H. B., Mironov, I., Talwar, K., & Zhang, L. (2016). Deep learning with differential privacy. Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, 308–318.
<https://www.tensorflow.org/>
- Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A., & Zou, J. (2019). Gradio: Hassle-free sharing and testing of ML models in the wild. arXiv preprint arXiv:1906.02569.
<https://gradio.app/>
- Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.
<https://opencv.org/>
- Nature. (2025). Scientific Reports article.
<https://www.nature.com/articles/s41598-025-99243-x>
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788.
<https://arxiv.org/abs/1506.02640>
- U.S. National Library of Medicine. (n.d.). PMC article.
<https://pmc.ncbi.nlm.nih.gov/articles/PMC12099366/>
- Ultralytics. (n.d.). YOLOv8 documentation.
<https://docs.ultralytics.com/models/yolov8/>
- Ultralytics. (n.d.). YOLO12 documentation.
<https://docs.ultralytics.com/models/yolo12/>
- Nontaphatfirm. (n.d.). Classroom Behavior Detector [GitHub repository].
<https://github.com/nontaphatfirm/Classroom-Behavior-Detector>
- Nontaphatfirm. (n.d.). Student behavior detector in classroom. Medium.
<https://medium.com/@nontaphatfirm/student-behavior-detector-in-classroom-20f2fe5>
- Kaggle. (n.d.). SCB-05 dataset.
<https://www.kaggle.com/datasets/shreyasudaya/scb-05-dataset>